版本: **1.0.5　2024 年 03 月**

AnyCloud39AV100
NNE 网络移植编程手册

© 2024 Guangzhou Anyka Microelectronics Co., Ltd.　All rights reserved.

声　　明

本手册的版权归广州安凯微电子股份有限公司所有，受相关法律法规的保护。未经广州安凯微电子股份有限公司的事先书面许可，任何人不得复制、传播本手册的内容。

本手册所涉及的知识产权归属广州安凯微电子股份有限公司所有（或经合作商授权许可使用），任何人不得侵犯。

本手册不对包括但不限于下列事项担保：适销性、特殊用途的适用性；实施该用途不会侵害第三方的知识产权等权利。

广州安凯微电子股份有限公司不对由使用本手册或执行本手册内容而带来的任何损害负责。

本手册是按当前的状态提供参考，随附产品或本书内容如有更改，恕不另行通知。

联　系　方　式

广州安凯微电子股份有限公司

地址：广州市黄埔区博文路 107 号安凯微电子 H 大厦

电话: (86)-20-3221 9000

传真: (86)-20-3221 9258

邮编: 510555

销售热线:

(86)-20-3221 9499

电子邮箱:

sales@anyka.com

主页:

http: //www. Anyka.com

Anyka Confidential For Giec Use Only

版 本 变 更 说 明

以下表格对于本文档的版本变更做一个简要的说明。版本变更仅限于技术内容的变更，不包括版式、格式、句法等的变更。

版本	说明	完成日期
V1.0.0	正式发布	2022 年 11 月
V1.0.1	1.增加 convert 工程部分生成文件的介绍。 2.simulate 工程中后处理函数从 NNE_Run 函数分离。	2022 年 12 月
V1.0.2	1.增加支持 concat 和 upsample 算子。	2023 年 02 月
V1.0.3	1.增加支持 element_wise_add、reshape；扩展 concat、pooling。 2.增加“环境搭建”、“上板运行”章节。	2023 年 04 月
V1.0.4	1. 卷积层约束放宽。 2. 算子输入输出个数上限改为 8。 3. 增加 split 和 concat 支持的维度。	2023 年 11 月
V1.0.5	1. 支持 MatMul/Gemm/Transpose，优化 Reshape。 2. NNE_INPUT_NORM_E 枚举成员增加 NNE_INPUT_NORM_U8，由 NNE 实现；NNE_INPUT_NORM_01 修改为 NNE 实现。 3. Convert 输出 network_output_blob 顺序同 onnx 一致。 4. Conv 约束放宽。 5. 支持单独的 AK_XRelu。 6. Quan 优化，FeatureMap 数值范围放宽，绝对值最大值由 128 增大到 1024。 7. Simulate 增加 NNE_Run_Interrupt 接口。	2024 年 03 月

Anyka Confidential For Giec Use Only

目录

1 NNE 概述 5
2 NNE 特性 5
2.1 NNE 网络层级联关系 5
2.2 基本级联层约束 6
2.2.1 卷积层约束 6
2.2.2 归一化层约束 7
2.2.3 激活层约束 7
2.2.4 池化层约束 8
2.3 基本级联层以外支持的算子 8
2.4 输入层预处理 10
2.5 输入数据格式 11
2.6 输出数据格式 11
3 NNE 开发工具 12
3.1 开发流程概述 12
3.2 准备测试/校准数据集 16
3.3 填写配置文件 17
3.4 转换模型为 onnx 19
3.5 写后处理 C 代码 22
3.6 环境搭建 24
3.6.1 环境配置 24
3.6.2 快速上手 24
3.7 convert 26
3.7.1 工具 26
3.7.2 输入文件 26
3.7.3 操作方法 26
3.7.4 输出文件 26
3.8 quan 36
3.8.1 量化方法介绍 36
3.8.2 工具 37
3.8.3 输入文件 37
3.8.4 操作方法 37
3.8.5 输出文件 37
3.9 check 38
3.9.1 工具 38
3.9.2 输入文件 39
3.9.3 操作方法 39
3.9.4 输出文件 39
3.10 simulate 40
3.10.1 工具 40
3.10.2 操作方法 40
3.10.3 输出文件 41

Anyka Confidential For Giec Use Only

3.11 上板运行 41
3.11.1 编译应用 demo 42
3.11.2 运行应用 demo 43

Anyka Confidential For Giec Use Only

1 NNE 概述

本文档介绍 NNE（Neural Network Engine）网络部署工具相关信息。

模型转换和量化步骤提供可执行文件，其他PC端工程提供源码。上板运行提供libak_nne库。

2 NNE 特性

2.1 NNE 网络层级联关系

NNE 一次可完成“卷积层+归一化层+激活层+池化层”的操作（下文称之为“基本级联层”），其中归一化层、激活层、池化层可以 bypass。归一化层和激活层的 bypass 通过设置参数来实现（硬件会计算，输出等于输入），池化层的 bypass 可以通过寄存器来设置，硬件不会计算。如果需要将卷积层 bypass，可通过设计卷积核参数使得输出等于输入来实现（硬件会计算）。

单独出现的 bn 算子会转换为卷积算子。

单独出现的 active 算子会增加恒等卷积来构成“基本级联层”。

单独出现的 pooling 算子会增加恒等卷积来构成“基本级联层”。

以上所说 conv，bn，active，pooling 算子有约束条件。

部分实现数据搬运的算子将转换为“基本级联层”。

其他算子由 CPU 计算。如果网络中包含较多其他算子，CPU 运算与 NNE 运算频繁切换，这样做效率很低，不推荐。

因此，为了高效的利用 NNE，建议用户将网络中的算子组合成 NNE 的“基本级联层”。组合方式如表 2-1 所示。

表 2-1 NNE 网络层级联组合表

1	conv+bn+active+pooling
2	covn+bn+active
3	conv+bn+pooling
4	conv+bn
5	conv+active+pooling
6	conv+active
7	conv+pooling
8	conv
9	bn+active+pooling
10	bn+active
11	bn+pooling
12	bn
13	active+pooling
14	active
15	pooling

Anyka Confidential For Giec Use Only

2.2 基本级联层约束

2.2.1 卷积层约束

表 2-2 NNE 对卷积层的约束

参数		参数解释	取值范围及约束	
input	input_w	input_img_width	input_w_padded=input_w+conv_left_pad ,取值范围 **[1, 2040]**. & (input_w+conv_left_pad+conv_right_pad)>=conv_kw .	
	input_h	input_img_height	**[1,2048]**. & (input_h+conv_top_pad+conv_bottom_pad)>=conv_kw.	
	input_ch	input_img_channel	if (conv_mode==1)，需满足表 2-4 限制。 if (conv_mode==2)，**[1, any]**。	
kernel	conv_mode	卷积模式	**[1,2]**	
	conv_kw	kernel_size	**[1, 3]**. & conv_kw==conv_kh & conv_kw<=(input_w+conv_left_pad+conv_right_pad) . & conv_kh<=(input_h+conv_top_pad+conv_bottom_pad).	
	conv_kch	kernel_input_channel	if(conv_mode==1), kernel_ch==input_ch. if(conv_mode==2),kernel_ch==1.	
	conv_kn	kerne_output_channel	**[1, any].**	
	conv_stride	卷积步长	**[1,2]**	当 kernel_w==1 时，需满足：conv_stride==1. & conv_left_pad==conv_right_pad==conv_top_pad==conv_bottom_pad==0 .
	conv_left_pad	卷积左边填充列数	**[0,1]**	
	conv_right_pad	卷积右边填充列数	**[0,1]**	
	conv_top_pad	卷积上边填充行数	**[0,1]**	
	conv_bottom_pad	卷积底边填充行数	**[0,1]**	

Anyka Confidential For Giec Use Only

表 2-3 输入图像宽度和输入图像通道数的最大值之间的约束关系

输入图像宽度	输入图像通道数的最大值
1-8	1024
9-16	512
17-32	256
33-64	128
65-128	64
129-256	32
257-512	16
513-1024	8
1025-2040	4

注意：表 2-3 中的输入图像宽度是指 input_w 与 conv_left_pad 之和。

其他说明

- 卷积模式有两种，普通卷积(conv_mode==1)和深度可分离卷积(conv_mode==2)，深度可分离卷积只支持“group==input_ch”的情况。
- 卷积 padding 需满足以下公式:

$$(input_w - conv_kw + conv_left_pad + conv_right_pad)\%conv_stride == 0$$
$$(input_h - conv_kw + conv_top_pad + conv_bottom_pad)\%conv_stride == 0$$

- 表 2-3 中输入图像宽度是指 input_w 与 conv_left_pad 之和。
- 普通卷积不满足表 2-3 时将拆分处理，拆分后表 2-3 扩展为表 2-4。

表 2-4 输入图像宽度和输入图像通道数的最大值之间的约束关系扩展

输入图像宽度	输入图像通道数的最大值
1-8	8192
9-16	4096
17-32	2048
33-64	1024
65-128	512
129-256	256
257-512	128
513-1024	64
1025-2040	32

2.2.2 归一化层约束

仅支持 batch norm 的归一化方式。工具处理过程默认会将卷积层和 BN 的参数融合，融合后在基本级联层中通过设计参数使得输出等于输入来实现 BN 层的 bypass，该参数由工具自动生成。

2.2.3 激活层约束

激活层可以 bypass，也可以支持 Relu/PRelu/LeakyRelu。如果 ONNX 模型中激活层是 Relu4/Relu6/Relu8，在 convert 和 quan 工具中不会报错，在量化时浮点推理采用 Relu4/Relu6/Relu8，获得量化方案。在定点推理或硬件上执行时，Relu4/Relu6/Relu8 操作将

Anyka Confidential For Giec Use Only

当做 Relu 来处理。激活层的 bypass 通过设计参数使得输出等于输入来实现，该参数由工具自动生成。

2.2.4 池化层约束

表 2-5 NNE 对池化层的约束

参数		参数解释	取值范围及限制
pooling	has_pool	0 表示关闭池化功能 1 表示打开池化功能	**[0,1]**
	pool_type	0 表示 AVERAGE_POOLING 1 表示 MAX_POOLING	**[0,1]**
	pool_kw	池化核尺寸	**[2,3]**. & pool_kh==pool_kw & pool_kw<=(conv_out_w+pool_right_pad). & pool_kh<=(conv_out_h+pool_bottom_pad).
	pool_stride	池化步长	**[1,2]**
	pool_right_pad	池化右边填充列数	**[0,1]**
	pool_bottom_pad	池化底边填充行数	**[0,1]**

其他说明

- 池化功能可以关闭。关闭后硬件不会计算。
- 池化 padding 仅支持 pool_right_pad 和 pool_bottom_pad；不支持 pool_left_pad、pool_top_pad。
- 池化 padding 仅支持填充“0”，不支持其他数值的填充。
- 池化有 padding 的情况下，padding 的值会参与运算。（onnx 中 AvgPool 的 count_include_pad 属性仅支持 True）。
- 池化不支持舍弃行或列，需满足：

$$(conv_output_w - pool_kw + pool_right_pad)\%pool_stride == 0$$
$$(conv_output_h - pool_kw + pool_bottom_pad)\%pool_stride == 0$$

其中 conv_output_w 和 conv_output_h 分别为卷积计算输出数据的宽和高。

2.3 基本级联层以外支持的算子

基本级联层之外的算子支持如下：

表 2-6 基本级联层之外的算子

序号	层类型	说明
1	Global_Pooling	全局池化（包括最大值池化和均值池化）。 由 CPU 处理。
2	Other_Pooling	除“基本级联层”中的池化和全局池化以外的池化。 由 CPU 处理。可能会转换为基本级联层由 NNE 处理。

Anyka Confidential For Gic Use Only

序号	层类型	说明
3	Flatten	axis 为 1 的维度变换。 由 CPU 处理。
4	Concat_Shuffle_Slice	concat+shuffle+slice 三个算子的组合（出现在 shufflenet_v2）。 由 CPU 处理。
5	Concat_Shuffle	concat+shuffle 两个算子的组合（出现在 shufflenet_v2）。 由 CPU 处理。
6	Concat	支持最大输入个数为 8。 axis 支持 c/h/w。 Concat_H 由 CPU 处理；当 Concat_W 中 w 不等时由 CPU 处理；其他将转为 Conv 由 NNE 处理。
7	Split	支持最大输入个数为 8。 axis 支持 c/h/w。 Split_H 由 CPU 处理；当 Split_W 中 w 不等时由 CPU 处理。其他将转为 Conv 由 NNE 处理。
8	Upsample	支持放大倍数为 2 的最近邻上采样（即 1 个值复制为 4 个值）。工具将拆分为 Resize_W 和 Resize_H 两步，Resize_W 由 CPU 处理，Resize_H 由 NNE 处理。有约束。
9	Reshape	支持[c,h,w]维度的变换。 由 CPU 处理。可能会和其他算子合并而优化。
10	Padding	axis 支持 c/h/w。 仅支持填充“零”。 由 CPU 处理。可能会和其他算子合并而优化。
11	Add	element_wise_add。 将转换为 conv 由 NNE 处理。
12	Slice	axis 支持 c/h/w。 由 CPU 处理。可能会和其他算子合并而优化。
13	Transpose	支持[c,h,w]维度的变换。 由 CPU 处理。可能会和其他算子合并而优化。
14	Gemm	仅支持 TransA=0。 将转换为“Transpose+Conv+Transpose”。其中 Transpose 由 CPU 处理，可能会和其他算子合并而优化；Conv 由 NNE 处理。
15	MatMul	是 Gemm 的特例，将被当做 Gemm 处理。

说明：

- Other_Pooling 中有一些特例可以转换为 NNE 支持的算子。例如 MaxPool((kernel==1)&&(stride==2)&&(pad==0)，出现在 Resnet_v2_50 中)将转换为 Conv，由 NNE 处理。例如 MaxPool((kw==kh)&&(kw%2==1)&&(stride_w==stride_h)&&(pad==0)&&(输入输出尺寸一致)&&(输入数据为非负数)，出现在 yolov5s 的 spp 结构中)，将转换为多层 3*3MaxPool，由 NNE 处理。
- Slice 中有一些特例可以转换为 NNE 支持的算子，由 NNE 处理。例如 yolox_nano 中 focus 模块。

Anyka Confidential For Client Use Only

2.4 输入层预处理

对输入图片的预处理指的是“减均值，除方差”的归一化处理。工具中支持三种方式：

```
typedef enum NNE_INPUT_NORM_E
{
    NNE_INPUT_NORM_ABS1  = 0x0,    //y=(x-128)/128;    //x:[0,255],y:[-1,1]
    NNE_INPUT_NORM_01  = 0x1,      //y=(x-0)/255;      //x:[0,255],y:[0,1]
    NNE_INPUT_NORM_U8  = 0x2       //y=(x-0)/1;        //x:[0,255],y:[0,255]
}NNE_INPUT_NORM_E;
```

方式 1 表示均值为[128,128,128]，方差为[128,128,128]。在浮点推理过程中，预处理函数内实现 y=(x-128)/128，其中 x∈[0,255]；y∈[-1,1]。在定点推理过程中，预处理函数内仅做数据拷贝或指针赋值，即预处理函数输出数据为 x，数据范围[0,255]。以该数据块作为输入的Conv层完成输入数据块的“归一化+量化”过程，实际操作是由NNE根据指令执行“-128”（减去 128）的操作，“归一化+量化”之后，数据范围[-128,127]，量化缩放因子被设置为 128。该 Conv 层的输出数据的量化缩放因子根据统计结果确定。

方式 1 中 NNE 根据指令执行“-128”即完成“归一化+量化”过程的原理解释如下：

假设输入数据为 X，预处理后数据为 Y，则

$$Y = \frac{X - mean}{std} * scale$$

满足以上条件时，公式可简化为

$$y = x - 128$$

方式 2 表示均值为[0,0,0]，方差为[255,255,255]。在浮点推理过程中，预处理函数内实现 y=(x-0)/255，其中 x∈[0,255]；y∈[0,1]。在定点推理过程中，预处理函数内仅做数据拷贝或指针赋值，即预处理函数输出数据为 x，数据范围[0,255]。方式 2 会构造一个 Conv 层，y=x/255 的计算过程转换为 y=((x-128)+128)/255=(x1+128)/255=x1/255+128/255，其中“x1=x-128”，由 NNE 根据指令执行“-128”实现；“y=x1/255+128/255” 则转为卷积操作，kernel=1/255，bias=128/255。以上两个步骤均在构造的 Conv 层中实现，其输入数据范围[0,255]，量化缩放因子被设置为 1；输出数据浮点范围[0,1]，量化缩放因子被设置为 128，定点数据范围[0,127]。

方式 3 表示均值为[0,0,0]，方差为[1,1,1]。在浮点推理过程中，预处理函数内实现 y=(x-0)/1=x，其中 x∈[0,255]；y∈[0,255]。在定点推理过程中，预处理函数内仅做数据拷贝或指针赋值，即预处理函数输出数据为 x，数据范围[0,255]。方式 3 会构造一个 Conv 层，y=x 的计算过程转换为y=(x-128)+128=x1+128，其中“x1=x-128”，由NNE根据指令执行“-128”实现；“y=x1+128” 则转为卷积操作，kernel=1，bias=128。以上两个步骤均在构造的 Conv 层中实现，其输入数据范围[0,255]，量化缩放因子被设置为 1；输出数据浮点范围[0,255]，量化缩放因子被设置为 1/2，定点数据范围[0,127]。

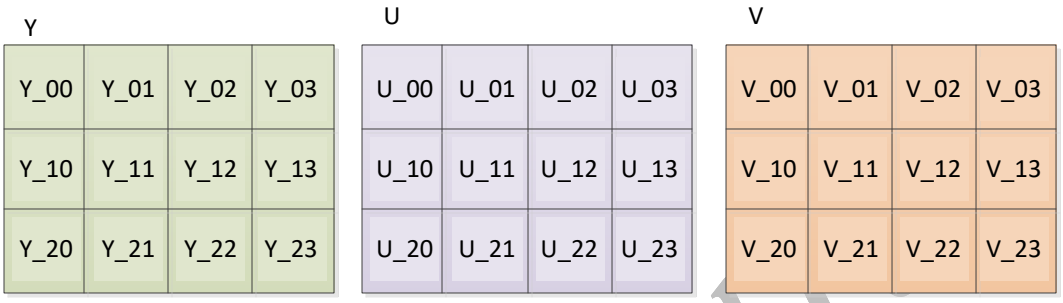
工具不支持以上方式以外的其他方式，因为在定点推理过程中，对每一张图都要做相同

Anyka Confidential For Gac Use Only

的“归一化+量化”操作，该操作为浮点计算，占用 cpu 资源大，一般难以承受。

2.5 输入数据格式

NNE 支持的输入数据格式有两种，分别为 YUV444 和 RGB888。输入数据在内存中的排列方式为[H, C, W]。图 2-1 以 YUV444 数据为例演示了数据排列方式。ISP 模块可直接输出该格式的图片数据。



存储顺序：
1. 先存plane_Y的line0,再存plane_U的line0,再存plane_V的line0.
2. 接着存plane_Y的line1，再存plane_U的line1，再存plane_V的line1.
3. 接着存plane_Y的line2，再存plane_U的line2，再存plane_V的line2.
即
Y_00, Y_01, Y_02, Y_03, U_00, U_01, U_02, U_03, V_00, V_01, V_02, V_03,
Y_10, Y_11, Y_12, Y_13, U_10, U_11, U_12, U_13, V_10, V_11, V_12, V_13,
Y_20, Y_21, Y_22, Y_23, U_20, U_21, U_22, U_23, V_20, V_21, V_22, V_23.

图 2-1 YUV444 数据在内存中的排列方式演示

2.6 输出数据格式

NNE 输出数据在内存中的排列方式为[H, C, W]，与输入数据的排列方式相同。

Anyka Confidential For Giec Use Only

3 NNE 开发工具

3.1开发流程概述

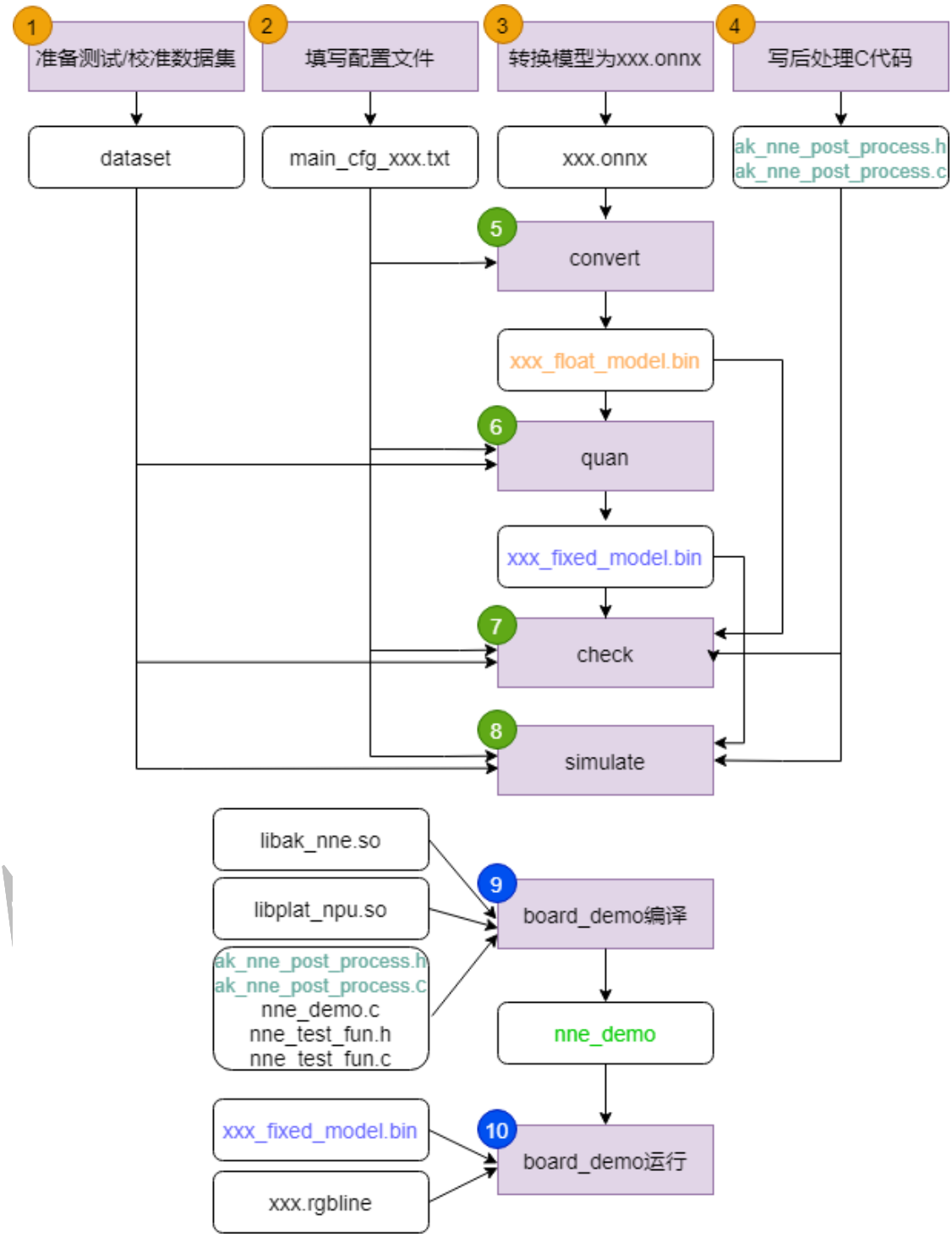


图 3-1 NNE 工具开发流程图

NNE 工具开发流程图中对于 convert/quan/check/simulate 工程的输出文件没有列出全部内容，补充如下：

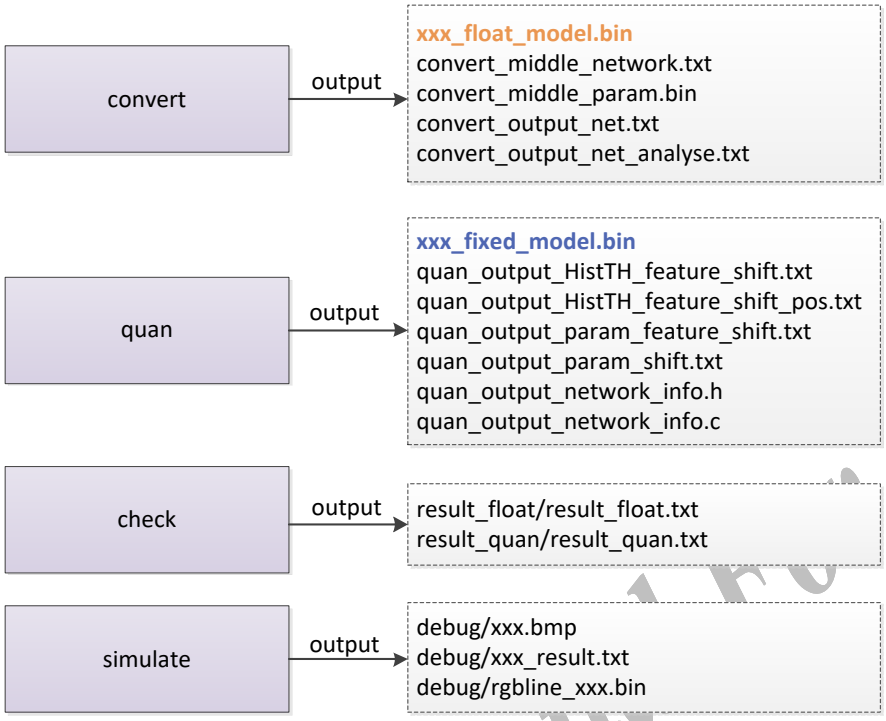


图 3-2 convert/quan/check/simulate 工程输出文件

流程共分为 10 步。使用的平台和功能描述如下：

表 3-1 NNE 工具开发流程步骤功能说明表

序号	步骤	平台	功能
1	准备测试/校准数据集	用户完成	测试/校准数据集后续将作为网络的输入数据，如有 crop/resize 等操作，在此步骤处理。网络的输入分辨率要求一致，不支持可变分辨率输入。
2	填写配置文件	用户完成	按照配置文件填写说明填写当前网络的配置信息。
3	转换模型为 xxx.onnx	用户完成	训练完导出该网络的 onnx 模型。 使用 pytorch/tensorflow 等框架中自带的导出工具。
4	写后处理 C 代码	用户完成	NNE 工具包对 onnx 模型的支持暂不完善，后处理部分较为复杂，请将后处理部分从 onnx 模型中剔除，使用 C 代码来实现。
5	convert	工具链使用环境	onnx 模型转换为 AK 浮点模型。
6	quan	工具链使用环境	AK 浮点模型转换为 AK 定点模型。
7	check	工具链使用环境	浮点模型的正向推理和定点模型的正向推理。 可对推理结果做进一步分析，判断是否满足精度要求。

Anyka Confidential For Giec Use Only

序号	步骤	平台	功能
			浮点推理结果用来验证 Convert 和后处理函数中的浮点部分是否正确。 定点推理结果用来验证 Quan 和后处理函数中的定点推理部分是否正确。
8	simulate	工具链使用环境	使用 libak_nne 库接口实现定点模型的正向推理。模拟开发板上运行网络的过程。
9	board_demo 编译	SDK 环境	编译上板运行的 demo。需加入 libak_nne 库和 libplat_npu 库、调整上板运行 demo 的主函数和后处理代码，编译为可执行程序。
10	board_demo 运行	开发板	运行以上可执行文件。需传入定点模型文件和图片数据。

特别说明

完整网络推理过程被分成三部分，预处理、主网络推理和后处理，如图 3-3 所示。

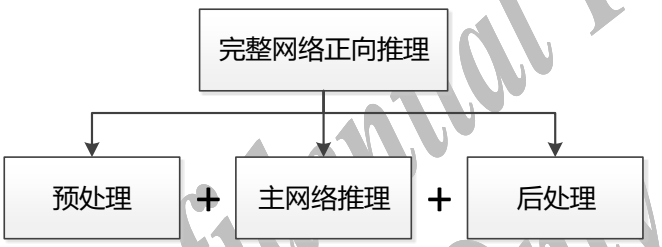


图 3-3 完整网络正向推理切分说明图

其中预处理部分包含输入图片缩放、裁剪、归一化等，建议使用 ISP 输出的第三通道数据作为网络输入且使用 NNE 支持的归一化方式，则预处理部分可略过。

主网络推理部分从网络输入层开始到特征层计算完毕。该部分主要由 NNE 加速器实现，不能使用 NNE 加速器实现的算子由 CPU 实现。

后处理部分对前步骤输出的特征层数据做处理输出最终结果。后处理部分由用户使用 C 函数实现，在 CPU 上执行，由用户调用。

为了说明主网络推理和后处理的切分方法，以 SSD 检测网络为例加以说明，如图 3-4 所示，主网络推理部分负责输出类别置信度和边界框偏移的特征金字塔，此例中是 4 层分类置信度层和 4 层边界框偏移层，共 8 层，即网络输出数量为 8。在后处理部分从特征金字塔原始数据中计算得出最终目标框个数及对应坐标，包括 confidence 与阈值对比做筛选、计算先验框信息、结合先验框和边界偏移得到真实坐标、NMS 等操作。

Anyka Confidential For Gree Use Only

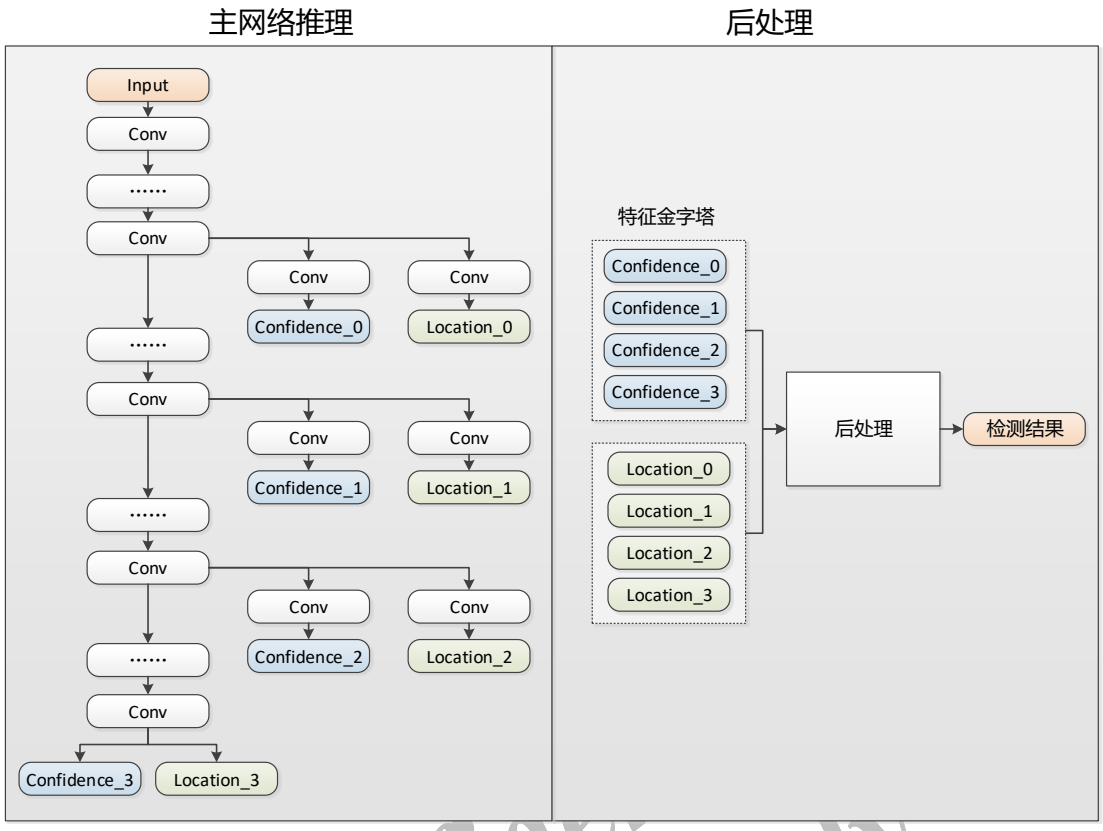


图 3-4 SSD 网络的主网络推理和后处理切分示意图

为了方便描述，下文中的文档结构根目录记为“nne_compile_tool”，工具包文件结构图如图 3-5。

nne_compile_tool/Tools 目录下存放 PC 端工具，包含 4 个步骤，分别是 Convert、Quan、Check、Simulate。Convert 目录下 convert 是可执行文件，将 onnx 模型转换为特定浮点模型；Quan 目录下 quan 是可执行文件，将浮点模型转换为定点模型；Check 路径下是浮点模型和定点模型正向推理需要的部分源码；Simulate 路径下是定点模型仿真需要的部分源码；Public 中的文件是四个步骤的公共文件。nne_compile_tool/Tools 目录下的文件是所有网络通用的文件，与具体网络无关。与网络相关的后处理函数及 check 步骤主函数、simulate 步骤主函数存放在 nne_compile_tool/Demos/xxx/vs_src/目录下。

nne_compile_tool/Demos/目录下存放了多个 demo，用来演示网络移植步骤。每个 demo 包含 onnx 文件、配置文件、后处理 c 文件、check 步骤主函数、simulate 步骤主函数等。nne_compile_tool/Demos/vs_src/目录下已存放该 demo 的 check 和 simulate 步骤的可执行文件。

nne_compile_tool/DataSet/路径下放置了几个 Demo 用到的校准/测试数据集。

nne_compile_tool/Docs/路径下放置了相关文档。

nne_compile_tool/上板验证/路径下存放在板端推理所需文件，包含 libak_nne 库、libak_npu 库、nne_demo、test_data、运行 nne_demo 的命令行说明。

nne_compile_tool/packages/目录下存放编译环境安装所需包。

Anyka Confidential Gec Use Only

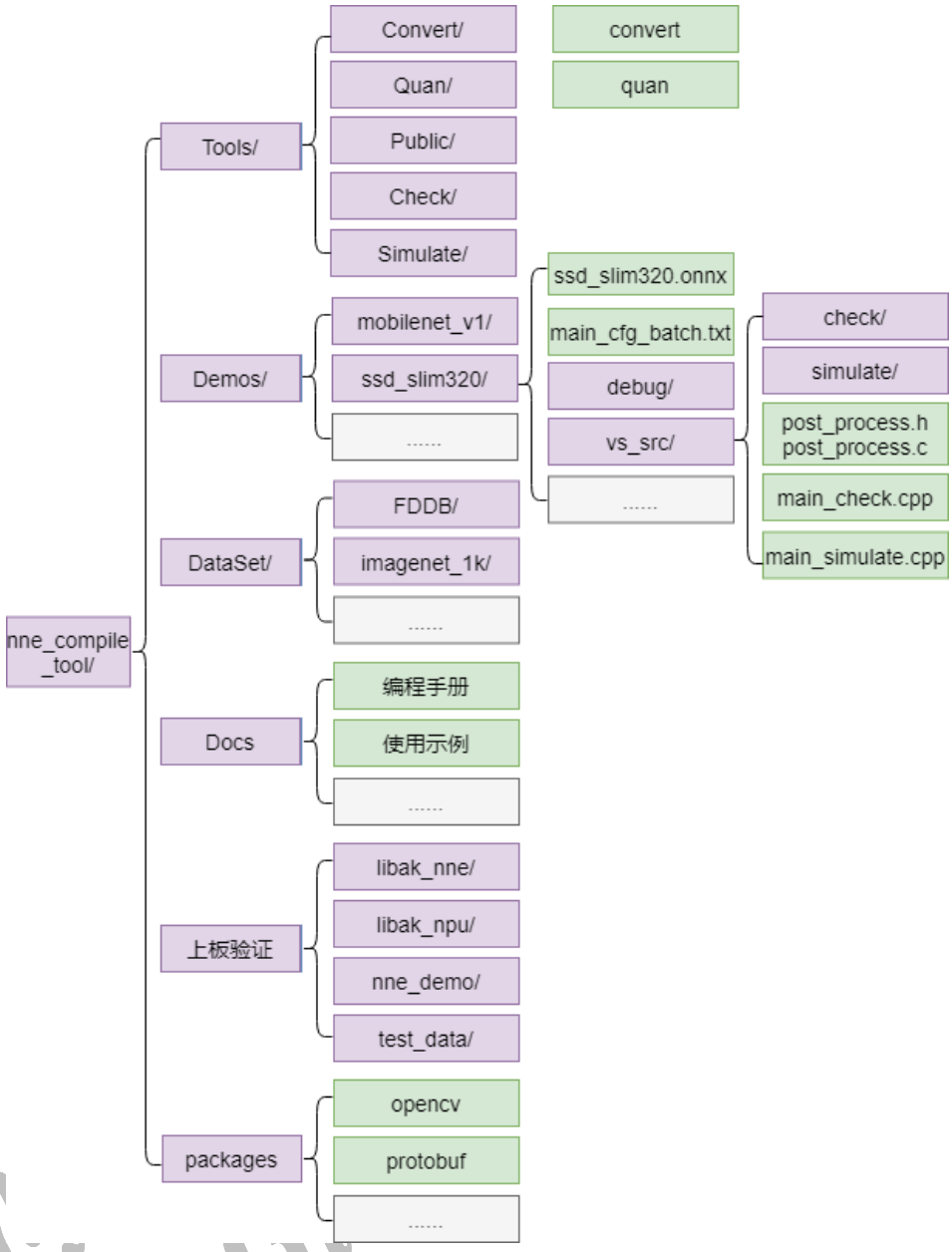


图 3-5 工具包文件结构图

3.2 准备测试/校准数据集

这一步由用户完成。

NNE 模块在实际应用时，使用 CPU 做图片预处理是低效率的，可直接使用 ISP 输出数据作为网络输入。在 quan/check/simulate 步骤中也要求输入图片分辨率一致，且可直接作为网络的输入，如果需要 resize/crop 等在这一步完成。

quan 步骤需要使用校准数据集来统计网络正向推理过程中特征值的数据分布从而得到量化方案。check 步骤需要测试数据集来统计浮点推理精度和定点推理精度。测试和校准数据集尽可能接近，可以使用相同的数据集，在 Demo 中选择测试数据集中的一部分作为校准数据集。simulate 步骤使用 check 相同的测试集。

3.3 填写配置文件

这一步由用户完成。

在 nne_compile_tool/Demos/目录下新建文件夹 my_net，用于存放用户网络相关文件。新建 main_cfg_my_net.txt 作为配置文件。

配置文件在 convert/quan/check/simulate 步骤共用一份。使用 txt 文件保存信息，每一行包含 2 两个字符串，分别为关键字和参数。其中关键字固定，参数随网络而异。具体格式如下：

表 3-2 配置文件填写说明表

行数	关键字	说明	示例
1	base_dir	文件存放目录	nne_compile_tool/Demos/mobilenetv1/
2	onnx_file	xxx.onnx 文件名。 绝对路径为：base_dir+onnx_file	mobilenetv1.onnx
3	input_w	输入图片宽度	224
4	input_h	输入图片高度	224
5	input_c	输入图片通道数	3
6	input_color	输入图片颜色格式。 枚举变量。支持： NNE_INPUT_COLOR_BGR NNE_INPUT_COLOR_RGB NNE_INPUT_COLOR_YUV	NNE_INPUT_COLOR_RGB
7	input_norm	输入图片归一化方式。 枚举变量。支持： NNE_INPUT_NORM_ABS1 NNE_INPUT_NORM_01 NNE_INPUT_NORM_U8	NNE_INPUT_NORM_ABS1
8	convert_float_model	convert 工程输出的浮点网络模型。	mobilenetv1_float_model.bin
9	task	网络分类。 枚举变量。支持： NNE_TASK_Classification NNE_TASK_Detection	NNE_TASK_Classification
10	dataset_dir	测试/校准数据存放路径	nne_compile_tool/DataSet/imagenet_1k/val_224/
11	dataset_list	测试/校准数据文件名列表 （绝对路径为 dataset_dir+filename）	nne_compile_tool/DataSet/imagenet_1k/val_224/filename_list.txt
12	quan_image_num	用于量化（校准）的图片数量。如果填 0 则取 filename_list 中的全部图片。	100
13	quan_method	feature_map 的量化方法。 枚举变量。支持 NNE_QUAN_METHOD_Hist_Symmetry	NNE_QUAN_METHOD_Hist_Symmetry

Anyka Confidential For Client Use Only

行数	关键字	说明	示例
		NNE_QUAN_METHOD_USR_SET 当使用 NNE_QUAN_METHOD_USR_SET 时，后面必须列出所有层的自定义量化参数。	
14	quan_hist_th	NNE_QUAN_METHOD_Hist_Symmetry 方法中的阈值。建议尝试 0.01/0.001/0.0001/0.00001 等，分别统计量化误差再做选择。	0.001
15	quan_fixed_model	quan 工程输出的定点网络模型。	mobilenetv1_fixed_model.bin
16	test_float_en	check 工程是否跑浮点正向推理。	AK_TRUE
17	test_quan_en	check 工程是否跑定点正向推理。	AK_TRUE
18	test_image_num	用于测试的图片数量。如果填 0 则取 filename_list 中的全部图片。	100
19	draw_result	是否画出目标框且保存图片	AK_FALSE
20	image_bmp	simulate 工程用到的单张测试图路径。绝对路径为：base_dir+image_bmp（用户准备）	debug/n01443537_ILSVRC2012_val_00048840.bmp
21	image_bin	simulate 工程生成的 image_bmp 对应的网络输入数据，按照 NNE 要求的数据格式。	mobilenet_v1_224.rgbline
22	pool_adjust	设置 pooling 算子调整幅度。可填写 0/1/2。	0
23	model_header	模型版本信息，填写 4 个值，最终会写入模型文件头 NNE_MODEL_HEADER 结构体中的 model_label 和 v_abc 成员。	0 1 2 3（表示 model_id=0,version=V1.2.3)

input_w/input_h/input_c 指网络模型推理过程中的输入分辨率，convert 步骤会使用此处设置的值，而非 onnx 中的值。对于同一个网络模型（onnx），如果想要尝试其他输入分辨率下的推理，需要修改 cfg 文件中的 input_w/input_h/input_c，然后重新执行模型转换步骤得到新的定点模型；修改 onnx 中的输入分辨率（或者在导出 onnx 时修改分辨率）是无效的。

关于 quan_method，建议用户首次使用时填写 NNE_QUAN_METHOD_Hist_Symmetry，运行 Quan 后生成的文件中包含“quan_output_HistTH_feature_shift.txt”，该文件统计了所有层 feature_map 的直方图分布，并列出量化参数。用户可在此基础上修改。第二次运行时，quan_method 填写 NNE_QUAN_METHOD_USR_SET，并将修改后的量化参数填写在 NNE_QUAN_METHOD_USR_SET 之后，格式参考 nne_compile_tool/Demos/resnet_v2_50_retrain/main_cfg_UsrSet.txt。

pool_adjust 设置 pooling 算子调整幅度。可填写 0 或 1 或 2。可能调整的属性包括：left_pad/right_pad/top_pad/bottom_pad/count_include_pad/ceil_mode。“0”表示不做任何调整，只要不满足“基本级联层”对 pooling 的约束则由 CPU 处理；“1”表示在确保输出分辨率不变的前提下做一部分调整来尽可能使用 NNE 处理而不是 CPU；“2”表示尽可能使用 NNE 处理而不是 CPU 处理，输出分辨率可能改变。用户可在精度和效率之间权衡。

Anyka Confidential For Gec Use Only

model_header 填写模型版本号信息，用于模型版本管理。后面填写 4 个值，假设用 i,a,b,c 表示，则模型文件的文件头 NNE_MODEL_HEADER 结构体中的 model_id 等于 i，v_abc 成员等于((a & 0xff) << 24) | ((b & 0xff) << 16) | ((c & 0xff) << 8)。

3.4 转换模型为 onnx

这一步由用户完成。

生成的 onnx 文件放入 nne_compile_tool/Demos/my_net/目录下。

工具包支持的 ONNX 算子如表 3-3 所示。工具包暂不支持表 3-3 以外算子，因此在转换成 onnx 模型时需简化模型，去掉不支持的算子后再转换。原则上将卷积、全连接层等交给 NNE 加速器运算，得到输出后通过后处理函数实现剩下的处理步骤。

遇到不支持的算子，可先尝试调整版本。例如 pytorch 转 onnx 时调用 torch.onnx.export，其传入的“opset_version”参数可做调整。例如使用 onnxsim.simplify。

表 3-3 工具支持的 onnx 算子及约束

算子	算子的参数约束	报错信息
Conv	● kernel_shape.size()==2（仅支持 Conv2D） ● auto_pad 为空或者“NOTSET”。 ● 需满足表 2-2 NNE 对卷积层的约束	● Error! Unsupported [Conv] :(kernel_shape.size() != 2) ● Error! Unsupported [Conv]: ((auto_pad!="")&&(auto_pad != 'NOTSET'))
BatchNormalization	● 支持对[n,c,h,w]数据的 c 维度或者[h,w]数据的 w 维度做批量归一化。	/
Relu	/	/
PRelu	/	/
LeakyRelu	/	/
Clip	● 仅支持[min,max]为[0,4]或者[0,6]或者[0,8]中的一种。 ● 当“Clip”出现在“Conv+Clip”组合中（此处的 Conv 可以是转换后得到的算子），“Clip”将被转换为“Relu4”或“Relu6”或“Relu8”。 ● 当出现单独的“Clip”，将被转换为“基本级联层”。	● Error! Unsupported [Clip]: none of([0,4] or [0,6] or [0,8])
AveragePool	● kernel_shape.size()==2（仅支持 AveragePool2D） ● auto_pad 为空或者“NOTSET”。	● Error! Unsupported [AveragePool] : (kernel_shape.size() != 2) ● Error! Unsupported [AveragePool]: ((auto_pad!="")&&(auto_pad != 'NOTSET'))
MaxPool	● kernel_shape.size()==2（仅支持 MaxPool2D） ● auto_pad 为空或者“NOTSET”。	● Error! Unsupported [MaxPool] : (kernel_shape.size() != 2) ● Error! Unsupported [MaxPool]: ((auto_pad!="")&&(auto_pad != 'NOTSET'))

Anyka Confidential For Gac Use Only

算子	算子的参数约束	报错信息
GlobalAveragePool	/	/
GlobalMaxPool	/	/
Flatten	● 仅支持输出尺寸[n,c,h,w]为[1,1,1,w]的情况。	● Error! Unsupported [Flatten]: axis=%d, ((axis<0) || (axis>4)) ● Error!compute_input_output_dimention:[layer_%d]:AK_Flatten:output_mat.h!=1
Concat	● 支持[C/H/W]维度。 ● 支持最多 8 输入。	● Error! Unsupported [Concat]: ((axis<0)||(axis>3))
Split	● 支持[C/H/W]维度。 ● split.size()==0 或者 output_size ● split[i]的值为非负数。	● Error! Unsupported [Split]: ((axis<0)||(axis>3)) ● Error! Unsupported [Split]: (split.size() != output_size) ● Error! Unsupported [Split]: (split[ii] <= 0)
Reshape	● 仅支持 allowzero==0，即 size 中不可出现“0”。 ● 仅支持 shape.size()<=4。 ● shape 中“-1”的数量不超过 1。	● Error! Unsopported [Reshape]:allowzero!=0 ● Error! Unsopported [Reshape]:shape.size()>4 ● Error! Unsopported [Reshape]: too much [-1] in shape
Softmax	● 当“Softmax”出现在网络输出层，则删除，并提醒用户将其放入后处理。否则报错。	● Error! [layer_line_%d] Softmax is not the last layer
Resize	● 仅支持：(mode==”nearest”)&&(h_scale==2)&&(w_scale==2)&&(align!=”align_corners”&&(sizes.empty())	● Error! Unsupported Resize mode! ● Error! Unsupported Resize scales! ● Error! Unsupported Resize sizes! ● Error! Unsupported Resize align_corners!
graph.input_size()	● 仅支持单个输入。	● Error! Unsupported input num.
Add	● 仅支持 with_scalar==0。 ● 仅支持 element_wise_add。 ● 如果 Add 输入不满足表 2-3 则拆分处理，最多拆 8 份。	● Error! Unsupported Add:(with_scalar != 0) ● Error! Unsupported AK_Add: group=%d
Pad	● 仅支持 mode==”constant”。 ● 仅支持 value==0。 ● 仅支持 pad_size==8。 ● 仅支持 C/H/W 维度的填充。	● Error! Unsupported Pad mode! (must be constant) ● Error! Unsupported Pad value! (must be 0)

Anyka Confidential For Giec Use Only

算子	算子的参数约束	报错信息
		● Error! Unsupported Pad value! (pad_size != 8) ● Error! Unsupported Pad: ((N_begin != 0) || (N_end != 0) || (C_begin!=0) || (C_end!=0))
Transpose	● 2<=perm.size()<=4。 ● 当 perm.size()==4 时，perm[0]必须为 0。	● Error! Unsupported [Transpose]: perm.size()=%d ● Error! Unsupported [Transpose]: (perm[0] != 0)
Gemm	● transA == 0。 ● 仅支持带 weight 的矩阵乘，不支持 2 个 feature_map 的矩阵乘	● Unsopported [Gemm]:(transA != 0)
MatMul	● 仅支持带 weight 的矩阵乘，不支持 2 个 feature_map 的矩阵乘	● Unsupported [MatMul]

说明：

- 当“Clip”出现在“Conv+Clip”组合中（此处的 Conv 可以是转换后得到的算子），且[min,max]为[0,4]或者[0,6]或者[0,8]中的一种，“Clip”将被转换为“Relu4”或“Relu6”或“Relu8”，和 Conv 组合成“基本级联层”。如果“Clip”单独出现，将被转换为“基本级联层”。量化工具中正向推理时使用 Relu4/Relu6/Relu8，量化后生成的定点模型中将改为“Relu”。在定点推理过程中除法用右移替代，因此量化参数 scale 等于 2 的幂次方，建议用户尝试 Relu4 以减少量化精度损失。
- “Flatten”仅支持输出尺寸[N,C,H,W]为[1,1,1,W]的情况。
- 支持“Concat”出现在“Concat+ShuffleChannel”或“Concat+ShuffleChannel+Split”组合中，或单独的“Concat”算子。前者由 CPU 一次处理完成。后者，当 concat_w 不满足 w 等分时由 CPU 处理；concat_h 由 CPU 处理；其他情况会转换为恒等卷积由 NNE 处理。Concat 支持最多 8 输入。
- Transpose 的 perm.size()需满足[2,4]之间，当 perm.size()==4 时需满足 perm[0]==0。即只处理[C,H,W]这三个维度的 Transpose。
- “Reshape”算子需满足 shape.size()<=4，且不支持“0”，且“-1”的个数不超过 1。如果 shape.size()==4，需满足计算后 N 维度为 1。Reshape 只处理[C,H,W]这三个维度的 Reshape。
- 当“Softmax”出现在网络的输出层，将删除且将提示用户将其放入后处理部分。
- “Resize”实现上采样，需满足表 3-4 中的参数约束。Resize 将被拆分为 resize_w 和 resize_h 两步，前者由 CPU 实现；后者将转换为恒等卷积由 NNE 实现.
- “Add”指 element_wise_add，当输入尺寸 input_w 和 input_ch 满足表 2-3，工具将转换为 NNE 可处理的算子。不满足表 2-3 的情况将拆分，最多拆分 8 份，拆分后需满足表 2-3.
- “Pad”算子支持[C/H/W]维度填充常量“0”，属性“pads”的个数支持 8。需注意其他框架转换为 onnx 时 pads 属性含义的变化，onnx 中 pads 的 8 个值分别表示 N_begin/C_begin/H_begin/W_begin/N_end/C_end/H_end/W_end 。其中 N_begin/N_end/C_begin/C_end 必须为 0。

Anyka Confidential For Gec Use Only

- **所有算子的输入数据和输出数据的维度均为[N,C,H,W]，且满足 N==1。不支持其他维度的处理。**

推荐的算子表达方式:

表 3-4 推荐的 onnx 算子表达方式

卷积	Conv
全连接	Conv
	Reshape+Gemm
	Flatten+Gemm
BN	BatchNormalization
激活	Relu/PRelu/LeakyRelu/Clip
池化	AveragePool/MaxPool
全局池化	GlobalAveragePool/GlobalMaxPool
按元素加	Add
Concat	Concat
Split	Split
上采样	Resize
矩阵乘法	MatMul/Gemm
维度交换	Transpose
尺寸变化	Reshape
ShuffleChannel	Reshape+Transpose+Reshape

3.5 写后处理 C 代码

浮点模型和定点模型不包含后处理，由用户实现后处理 C 代码。该部分代码在 PC 端工具链的 check/simulate 工程以及上板验证时需要用到，且在 main 函数中调用。该部分代码实现自由，为方便描述，本文将其写入 ak_nne_post_process.h 和 ak_nne_post_process.c 两个文件，主要由 nne_post_process 函数实现。

在后处理函数中，请分别实现浮点处理和定点处理。在使用 PC 端工具时，需要同时实现浮点处理和定点处理。经过 PC 端工具验证无误后，部署到硬件上时可以将浮点推理部分注释掉。后处理的验证在 check 工程中完成。

check 工程和 simulate 工程均用到这两个文件，用户只需实现一次，然后拷贝使用。

工具包中给出了 Demo 的后处理函数，以 ssd_mobilenetv1 为例，在 nne_compile_tool/Demos/ssd_mobilenetv1/vs_src/ 路径下存放了 ak_nne_post_process.h 和 ak_nne_post_process.c。其中数据结构和接口函数（定点推理）说明如下：

【数据结构】

```
typedef struct Param_Intput_S
{
    float confidence_threshold;
    float iou_threshold;
}Param_Intput_S;
```

Anyka Confidential For Gitec Use Only

```
typedef struct OBJECT_INFO
{
    int label;
    int confidence;
    NNE_RECT_S box;
}OBJECT_INFO;

#define MAX_NUM_OF_BOX    20
typedef struct Param_Output_S
{
    int obj_num;
    OBJECT_INFO obj_info[MAX_NUM_OF_BOX];
}Param_Output_S;
```

【函数接口】

```
int post_process_int(NNE_BLOB_VECTOR_S *input_blobs, NNE_BLOB_VECTOR_S *output_blobs, Param_Intput_S input_param, Param_Output_S *output_param);
```

input_blobs 是网络输入，input_blobs->blob_cnt 表示输入个数。output_blobs 为主网络的输出（参考 3.1 节关于主网络和后处理的说明），output_blobs->blob_cnt 表示输出个数。

blob 的详细信息保存在 NNE_INPUT_OUTPUT_BLOB_INFO_S 结构体中，包含数据类型、数据排列方式、虚拟地址、物理地址、数据量、宽、高、通道数、相比对真实值的放大倍数。

对于多输出的网络，output_blob 的排列顺序与 onnx 相同。在 convert_output_net.txt 中分别打印了 network_input_blob 和 network_output_blob 的信息。以 yolov5_s_retrain 为例，nne_compile_tool/Demos/yolov5_s_retrain/convert_output_net.txt 文件的最下方：

```
network_input_blob:
name["images"]    blob_idx[0]    buf_idx[0]    w[640]   h[384]   c[3]
network_output_blob:
name["383"]       blob_idx[122] buf_idx[2]    w[80]    h[48]    c[21]
name["427"]       blob_idx[123] buf_idx[3]    w[40]    h[24]    c[21]
name["471"]       blob_idx[124] buf_idx[1]    w[20]    h[12]    c[21]
```

该网络输入 blob 只有一个，名字为“images”，所属 blob 的索引为 0，宽度为 640，高度为 384，通道数为 3。该网络的主网络输出有 3 个 blob，每一行列出一个 blob 的信息，分别对应 output_blobs->array_blobs[0]、output_blobs->array_blobs[1]、output_blobs->array_blobs[2]。如果某个 blob 信息和 onnx 有差异，可能是算子做了转换，例如“MatMul”将被转换为“Transpose+Conv+Transpose”，其中第二个“Transpose”满足“是输出层算子且使用 CPU 处理”的条件将被从主网络中排除，用户需在后处理函数中处理。在 convert 步骤的 log 中会提醒用户 output_blob 名字的改变。

input_param 保存后处理需要用到的阈值等信息。

output_param 保存后处理输出的目标框信息。

Confidential For Internal Use Only

3.6 环境搭建

NNE 开发流程中 convert、quan、check、simulate 步骤需在 PC 端仿真环境中编译运行。

3.6.1 环境配置

系统版本：Ubuntu 18.04.6 LTS

需要安装的基本工具有：g++、gcc、gdb、make、cmake。第三方依赖库有：protobuf、opencv。安装包都在 nne_compile_tool/packages/目录下，不需要联网，可以离线安装。

- 安装 g++、gcc、gdb、make

以上工具包依次安装，安装命令为 dpkg -i *.deb，需 sudo 权限，例如安装 g++命令如下：

```
cd g++/
dpkg -i *.deb
```

- 安装 cmake

cmake 采用源码安装，安装步骤如下：

```
cd cmake-3.20.0/
./bootstrap
make
make install
```

运行上面的命令无任何报错即安装成功，可以用 cmake --version 查看版本号。

注意，Public、Check、Simulate 这三个文件夹下均有一个“CMakeLists.txt”，编译时需要，请勿删除。

- 拷贝 opencv 和 protobuf 的 include、lib 到系统目录

opencv3.4.1 和 protobuf3.4.0 提供已编译好的 linux 版本，需分别将 include 文件和 lib 文件拷贝到系统目录：

```
cp -r nne_compile_tool/packages/opencv_3_4_1/include/opencv    /usr/include/
cp -r nne_compile_tool/packages/opencv_3_4_1/include/opencv2    /usr/include/
cp -r nne_compile_tool/packages/opencv_3_4_1/lib/*   /usr/lib/
cp -r nne_compile_tool/packages/protobuf_3_4_0/include/google       /usr/include/
cp -r nne_compile_tool/packages/protobuf_3_4_0/lib/*       /usr/lib/
```

3.6.2 快速上手

在 cnn_compile_tool/Demos/路径下，工具包提供了多个 demo 供用户参考。

在 cnn_compile_tool/Tools/路径下，包含 convert，quan，check，simulate 四个步骤的可执行文件或源码。其中 convert 和 quan 提供可执行文件，check 和 simulate 提供源码。

以 mobilenetv1 为例，若需要先验证 cnn_compile_tool/Demos 下提供的 demo 是否正常运行，首先修改 cnn_compile_tool/Demos/mobilenet_v1/目录下的 main_cfg_batch.txt 配置文件（主要修改 demo 和数据集的绝对路径，即 base_dir，dataset_dir 和 dataset_list 这三项对应的路径），然后在 cnn_compile_tool/Demos/mobilenet_v1/目录下执行如下相应的步骤：

- convert

在 cnn_compile_tool/Demos/mobilenet_v1/ 目录下执行：

```
../../Tools/Convert/convert    main_cfg_batch.txt
```

执行完后会在 cnn_compile_tool/Demos/mobilenet_v1/目录下生成 mobilenet_v1 的浮点推理模型文件 mobilenet_v1_float_model.bin 及 convert 步骤产生的其他文件。

Anyka Confidential For Internal Use Only

- quan

在 cnn_compile_tool/Demos/mobilenet_v1/ 目录下执行：

```
../../Tools/Quan/quan  main_cfg_batch.txt
```

会在 cnn_compile_tool/Demos/mobilenet_v1/目录下生成 mobilenet_v1 的定点推理模型文件 mobilenet_v1_fixed_model.bin 及 quan 步骤产生的其他文件。

- check

在 cnn_compile_tool/Demos/mobilenet_v1/vs_src/目录下已存放可执行文件“check”，如果需要自己生成可执行文件“check”，按如下步骤操作：

将 cnn_compile_tool/Demos/mobilenet_v1/vs_src/ 目录下的 ak_nne_post_process.h，ak_nne_post_process.c 和 main_check.cpp 三个文件拷贝到 cnn_compile_tool/Tools/Check/目录下，执行以下步骤：

```
cd  cnn_compile_tool/Tools/Check/
mkdir build
cd build
cmake  ..
make
```

在 cnn_compile_tool/Tools/Check/build/目录下生成可执行文件：check。将其拷贝到 cnn_compile_tool/Demos/mobilenet_v1/vs_src/目录下，执行：

```
./check ../main_cfg_batch.txt
```

执行 check 步骤后会在 cnn_compile_tool/Demos/mobilenet_v1/目录下生成 mobilenet_v1 模型的浮点推理结果和定点推理结果，该结果保存在目录 cnn_compile_tool/Demos/mobilenet_v1/result_float/ 和 cnn_compile_tool/Demos/mobilenet_v1/result_quan/下。

删除 cnn_compile_tool/Tools/Check/目录下当前步骤拷贝的文件和生成的文件。

- simulate

在 cnn_compile_tool/Demos/mobilenet_v1/vs_src/目录下已存放可执行文件“simulate”，如果需要自己生成可执行文件“simulate”，按如下步骤操作：

将 cnn_compile_tool/Demos/mobilenet_v1/vs_src/ 目录下的 ak_nne_post_process.h，ak_nne_post_process.c 和 main_simulate.cpp 三个文件拷贝到 cnn_compile_tool/Tools/Simulate/目录下，执行以下步骤：

```
cd  cnn_compile_tool/Tools/Simulate/
mkdir build
cd build
cmake  ..
make
```

在 cnn_compile_tool/Tools/Simulate/build/目录下生成可执行文件：simulate。将其拷贝到 cnn_compile_tool/Demos/mobilenet_v1/vs_src/目录下，执行：

```
./simulate ../main_cfg_batch.txt
```

运行结束后在 cnn_compile_tool/Demos/mobilenet_v1/debug 目录下生成 mobilenetv1 模型的单张仿真结果。

删除 cnn_compile_tool/Tools/Simulate/目录下当前步骤拷贝的文件和生成的文件。

用户移植自研网络的步骤：

Anyka Confidential Giec Use Only

参考 cnn_compile_tool/Demos 文件夹下的目录架构，在 cnn_compile_tool/Demos/目录下新建一个文件夹 test_net（举例说明，文件夹名可自定义）；

训练好网络，并导出 onnx 模型文件：test_net.onnx（举例说明，文件名可自定义），拷贝至 cnn_compile_tool/Demos/test_net/目录下；

准备好校准和测试数据集；

新建配置文件 main_cfg.txt 并填写，或从其他 Demo 中拷贝后修改内容，存放在 cnn_compile_tool/Demos/test_net/目录下；

实现后处理 C 代码、check 和 simulate 步骤的主函数，或从其他 Demo 中拷贝后修改内容，暂存于 cnn_compile_tool/Demos/test_net/vs_src/目录下；

按照以上 Demo 操作示例完成 convert、quan、check、simulate 步骤。

3.7 convert

onnx 模型转换为 AK 浮点模型。

3.7.1 工具

提供可执行文件，存放于：nne_compile_tool/Tools/Convert/convert。

3.7.2 输入文件

main 函数需要传入一个参数，该参数为配置文件名，一般命名为”main_cfg_xxx.txt”，其中“xxx”取网络名。文件名可以修改。main 函数内会对该文件做解析。

其他输入文件已在配置文件中列出，只有一个，是 onnx 模型文件。

3.7.3 操作方法

命令行：

```
./convert  main_cfg_test_net.txt
```

请根据实际情况调整文件路径。

3.7.4 输出文件

convert 工具输出 4 个文件，如表 3-5 所示。

表 3-5 Convert 工具输出文件列表

序号	文件名	说明	用途
1	xxx_float_model.bin	AK 浮点模型（包含网络结构信息和参数）	quan 工程的输入文件
2	convert_middle_network.txt convert_middle_param.bin	convert 工程从 onnx 解析出的算子信息和参数。	参考信息
3	convert_output_net.txt	AK 浮点模型中的网络结构信息。	参考信息
4	convert_output_net_analyse.txt	网络信息统计，包含 MAC 数量、读写数据量等。	参考信息

xxx_float_model.bin 包含三部分信息，分别为文件头、网络结构信息、参数，在 quan 工程中作为输入文件。

经过 convert 工程，从 onnx 中提取出的算子变换名称，并打印有效信息到

Anyka Confidential For Cisc Use Only

convert_middle_network.txt 中。表 3-6 是 onnx 算子与“AK”开头算子名的对应关系。convert_middle_network.txt 文件第一行表示输入节点数量和名字，第二行表示输出节点数量和名字，第三行表示网络层数量，从第四行开始每一行表示一层信息，具体信息如表 3-6 所示。

表 3-6 onnx 算子与 ak 算子对应关系

onnx_op	ak_op	运算载体	Demo 网络举例
网络输入	AK_Input		mobilenet_v1
Conv	AK_Convolution	NNE	mobilenet_v1
Conv	AK_ConvolutionDepthWise	NNE	mobilenet_v1
BatchNormalization	AK_BatchNorm	NNE	shufflenet_v2
Relu	AK_Relu	NNE	shufflenet_v2
PRelu	AK_PRelu	NNE	/
LeakyRelu	AK_LeakyRelu	NNE	/
Clip	AK_Clip	NNE	mobilenet_v1
AveragePool/ MaxPool	AK_Pooling	NNE	yolov5_s_retrain
		CPU	shufflenet_v2
GlobalAveragePool/ GlobalMaxPool	AK_Global_Pooling	CPU	mobilenet_v1
网络分支	AK_Branch	/	ssd_slim320.onnx
Flatten	AK_Flatten	CPU	/
Concat	AK_Concat	CPU	shufflenet_v2
		NNE	yolo_x_nano_retrain
Slice	AK_Slice	NNE	resnet20_cifar10 yolo_x_nano_retrain
	AK_Slice	CPU	/
Split	AK_Split	NNE	
		CPU	shufflenet_v2
Reshape	AK_Reshape	CPU	mobilenet_v1
Softmax	AK_Softmax	/	mobilenet_v1
“Reshape+Transpose+Reshape”组合	AK_ShuffleChannel	CPU	shufflenet_v2
Resize	AK_Upsample	NNE+CPU	yolo_x_nano_retrain
Add	AK_Add	NNE	mobilenet_v2_tf
Pad	AK_Padding	CPU	/
Gemm	AK_Gemm	NNE+CPU	shufflenet_v2
MatMul	AK_Gemm	NNE+CPU	
Transpose	AK_Transpose	CPU	/

说明：

AveragePool/MaxPool 算子根据其属性及输入信息，符合 NNE“基本级联层”约束的将被设置为 AK_Pooling，其他设置为 AK_Other_Pooling。AK_Other_Pooling 满足某些条件时会转换为“基本级联层”。

Anyka Confidential For Giec Use Only

Concat 算子如果出现在“Concat+ShuffleChannel”或“Concat+ShuffleChannel+Split”组合中将由 CPU 一次处理完；单独的“Concat”算子在一些情况下会转换为“基本级联层”由 NNE 实现。

MatMul 作为 Gemm 的特例，统一按 AK_Gemm 处理。AK_Gemm 会转换为“Transpose+Conv+Transpose”，其中 Transpose 由 CPU 实现，Conv 由 NNE 实现。一些情况下 Transpose 会被优化。

Reshape 或 Flatten 由 CPU 处理。一些情况下会被优化。

Pad 算子经过分析后可能和 Conv 或 AveragePool/MaxPool 合并。

表 3-7 convert_middle_network.txt 中 AK 算子信息

0	AK_Input	AK_Convolution	AK_ConvolutionDepthWise	AK_InnerProduct	AK_BatchNorm
1	node.name	node.name	node.name	node.name	node.name
2	node.input_size	node.input_size	node.input_size	node.input_size	node.input_size
3	node.output_size	node.output_size	node.output_size	node.output_size	node.output_size
4	node.name	input_name[0, input_size-1]	input_name[0, input_size-1]	input_name[0, input_size-1]	input_name[0, input_size-1]
5		output_name[0, output_size-1]	output_name[0, output_size-1]	output_name[0, output_size-1]	output_name[0, output_size-1]
6		filter_num	filter_num	weight.dims(1)	filter_num
7		kernel_shape[0]	kernel_shape[0]	weight.dims(0)	
8		kernel_shape[1]	kernel_shape[1]	weight_size	
9		dilations[0]	dilations[0]	bias_size	
10		dilations[1]	dilations[1]		
11		strides[0]	strides[0]		
12		strides[1]	strides[1]		
13		pads[0]-left	pads[0]-left		
14		pads[2]-right	pads[2]-right		
15		pads[1]-top	pads[1]-top		
16		pads[3]-bottom	pads[3]-bottom		
17		group	group		
18		weight_size	weight_size		
19		bias_size	bias_size		

表 3-7 续表 1

0	AK_Relu	AK_PRelu	AK_LeakyRelu	AK_Clip	AK_Pooling
1	node.name	node.name	node.name	node.name	node.name
2	node.input_size	node.input_size	node.input_size	node.input_size	node.input_size
3	node.output_size	node.output_size	node.output_size	node.output_size	node.output_size
4	input_name[0, input_size-1]	input_name[0, input_size-1]	input_name[0, input_size-1]	input_name[0, input_size-1]	input_name[0, input_size-1]

Anyka Confidential For Client Use Only

5	output_name[0, output_size-1]	output_name[0, output_size-1]	output_name[0, output_size-1]	output_name[0, output_size-1]	output_name[0, output_size-1]
6		num_slope	alpha	min	pool_type
7				max	kernel_shape[0]
8					kernel_shape[1]
9					strides[0]
10					strides[1]
11					pads[0]-left
12					pads[2]-right
13					pads[1]-top
14					pads[3]-bottom
15					ceil_mode
16					count_include_pad

表 3-7 续表 2

0	AK_Global_Pooling	AK_Flatten	AK_Concat	AK_Slice	AK_Split
1	node.name	node.name	node.name	node.name	node.name
2	node.input_size	node.input_size	node.input_size	node.input_size(1)	node.input_size
3	node.output_size	node.output_size	node.output_size	node.output_size(2)	node.output_size
4	input_name[0, input_size-1]	input_name[0, input_size-1]	input_name[0, input_size-1]	input_name[0, input_size-1]	input_name[0, input_size-1]
5	output_name[0, output_size-1]	output_name[0, output_size-1]	output_name[0, output_size-1]	output_name[0, output_size-1]	output_name[0, output_size-1]
6	pool_type		axis	group	axis
7				size_per_group	size_each_group

表 3-7 续表 3

0	AK_Reshape	AK_Softmax	AK_ShuffleChannel	AK_Upsample	AK_Add
1	node.name	node.name	node.name	node.name	node.name
2	node.input_size	node.input_size	node.input_size	node.input_size	node.input_size
3	node.output_size	node.output_size	node.output_size	node.output_size	node.output_size
4	input_name[0, input_size-1]	input_name[0, input_size-1]	input_name[0, input_size-1]	input_name[0, input_size-1]	input_name[0, input_size-1]
5	output_name[0, output_size-1]	output_name[0, output_size-1]	output_name[0, output_size-1]	output_name[0, output_size-1]	output_name[0, output_size-1]
6	shape[0,size-1]	axis	group		

Anyka Confidential For Giec Use Only

7			reverse		

表 3-7 续表 4

0	AK_Padding	AK_Gemm	AK_Transpose		
1	node.name	node.name	node.name		
2	node.input_size	node.input_size	node.input_size		
3	node.output_size	node.output_size	node.output_size		
4	input_name[0, input_size-1]	input_name[0, input_size-1]	input_name[0, input_size-1]		
5	output_name[0, output_size-1]	output_name[0, output_size-1]	output_name[0, output_size-1]		
6	pad_left	dim[0]	perm.size		
7	pad_right	dim[1]	perm[]		
8	pad_top	bias_size			
9	pad_bottom	weight			
10		bias			

表 3-8 工具支持的 AK 算子及算子约束

算子	算子的参数约束	报错信息
file	convert_middle_network.txt 前三行 ● 第 0 行表示网络输入节点个数及名字 ● 第 1 行表示网络输出节点个数及名字 ● 第 2 行表示算子总数。	● Error! [convert_middle_network.txt]: Line_0 info error. ● Error! [convert_middle_network.txt]: Line_1 info error. ● Error! [convert_middle_network.txt]: Line_2 info error.
AK_Input	● 格式需满足表 3-7 ● 输入个数为 1，输出个数为 1。	● Error! Unsupported AK_Input! Format error. ● Error! Unsupported AK_Input! (layer.input_num != 0) || (layer.output_num != 1)
AK_Convolution	● 格式需满足表 3-7。 ● 输入个数为 1，输出个数为 1。 ● 需满足表 2-2 NNE 对卷积层的约束。	● Error! Unsupported AK_Convolution! Format error. ● Error! Unsupported AK_Convolution! (layer.input_num != 1) || (layer.output_num != 1) ● Error! Unsupported AK_Convolution! kernel_w*kernel_h*kernel_ch*filter_num!=weight_size ● Error! Unsupported AK_Convolution! ((kernel_w != kernel_h) || (kernel_w < 1) || (kernel_w > 3))

Anyka Confidential For Giec Use Only

算子	算子的参数约束	报错信息
		● Error! Unsupported AK_Convolution! ((dilate_w != dilate_h) || (dilate_w!=1)) ● Error! Unsupported AK_Convolution! ((stride_w != stride_h) || (stride_w<1) || (stride_w>2)) ● Error! Unsupported AK_Convolution! ((pad_left > 1) || (pad_left < 0)) ● Error! Unsupported AK_Convolution! ((pad_right > 1) || (pad_right < 0) ● Error! Unsupported AK_Convolution! ((pad_top > 1) || (pad_top < 0)) ● Error! Unsupported AK_Convolution! ((pad_bottom > 1) || (pad_bottom < 0)) ● Error! Unsupported AK_Convolution! (group != 1) ● Error! Unsupported AK_Convolution! ((bias_size != 0) && (bias_size != filter_num))
AK_ConvolutionDepthWise	● 格式需满足表 3-7。 ● 输入个数为 1，输出个数为 1。 ● 需满足表 2-2 NNE 对卷积层的约束。	● Error! Unsupported AK_ConvolutionDepthWise! Format error. ● Error! Unsupported AK_ConvolutionDepthWise! ((layer.input_num != 1) || (layer.output_num != 1)) ● Error! Unsupported AK_ConvolutionDepthWise! ((kernel_w != kernel_h) || (kernel_w < 1) || (kernel_w > 3)) ● Error! Unsupported AK_ConvolutionDepthWise! ((dilate_w != dilate_h) || (dilate_w != 1)) ● Error! Unsupported AK_ConvolutionDepthWise! ((stride_w != stride_h) || (stride_w < 1) || (stride_w > 2)) ● Error! Unsupported AK_ConvolutionDepthWise! ((pad_left > 1) || (pad_left < 0)) ● Error! Unsupported AK_ConvolutionDepthWise! ((pad_right > 1) || (pad_right < 0)) ● Error! Unsupported AK_ConvolutionDepthWise! ((pad_top > 1) || (pad_top < 0))

Anyka Confidential For Giec Use Only

算子	算子的参数约束	报错信息
		● Error! Unsupported AK_ConvolutionDepthWise! ((pad_bottom > 1) || (pad_bottom < 0)) ● Error! Unsupported AK_ConvolutionDepthWise! (group != filter_num) ● Error! Unsupported AK_ConvolutionDepthWise! ((kernel_w*kernel_h*1* filter_num) != weight_size) ● Error! Unsupported AK_ConvolutionDepthWise! ((bias_size != 0) & (bias_size != filter_num))
AK_BatchNorm	● 格式需满足表 3-7。 ● 输入个数为 1，输出个数为 1。 ● BN 的参数个数与前面 Conv 的 channel 个数相同或者与前面 Gemm 的 width 相同。	● Error! Unsupported AK_BatchNorm! Format error. ● Error! Unsupported AK_BatchNorm! ((layer.input_num != 1) || (layer.output_num != 1)) ● Error! Unsupported AK_BatchNorm! (Size error)
AK_Relu	● 格式需满足表 3-7。 ● 输入个数为 1，输出个数为 1。	● Error! Unsupported AK_Relu! Format error. ● Error! Unsupported AK_Relu! ((layer.input_num != 1) || (layer.output_num != 1))
AK_PRelu	● 格式需满足表 3-7。 ● 输入个数为 1，输出个数为 1。	● Error! Unsupported AK_PRelu! Format error. ● Error! Unsupported AK_PRelu! ((layer.input_num != 1) || (layer.output_num != 1))
AK_LeakyRelu	● 格式需满足表 3-7。 ● 输入个数为 1，输出个数为 1。	● Error! Unsupported AK_LeakyRelu! Format error. ● Error! Unsupported AK_LeakyRelu! ((layer.input_num != 1) || (layer.output_num != 1))
AK_Clip	● 格式需满足表 3-7。 ● 输入个数为 1，输出个数为 1。 ● 仅支持 Relu4 或 Relu6 或 Relu8。	● Error! Unsupported AK_Clip! Format error. ● Error! Unsupported AK_Clip! ((layer.input_num != 1) || (layer.output_num != 1)) ● Error! Unsupported AK_Clip! None of [0,4] of [0,6] or [0,8]

Anyka Confidential For Giec Use Only

算子	算子的参数约束	报错信息
AK_Pooling	● 格式需满足表 3-7。 ● 输入个数为 1，输出个数为 1。 ● 需满足表 2-4 NNE 对池化层的约束。（如果不满足则根据配置文件中的 pool_adjust 决定是否转为 AK_Other_Pooling）	● Error! Unsupported AK_Pooling! Format error! ● Error! Unsupported AK_Pooling! (layer.input_num != 1) || (layer.output_num != 1) ● Error! Unsupported AK_Pooling! (pool_type < 0 || pool_type>1) ● Warning! [layer_%d] Unsupported AK_Pooling! kw=[%d] kh=[%d].Set new type: NNE_OP_TYPE_Other_Pooling ● Warning! [layer_%d] Unsupported AK_Pooling! stride_w=[%d] stride_h=[%d].Set new type: NNE_OP_TYPE_Other_Pooling ● Warning! [layer_%d] Unsupported AK_Pooling! pad_left=[%d].Set new type: NNE_OP_TYPE_Other_Pooling ● Warning! [layer_%d] Unsupported AK_Pooling! pad_top=[%d].Set new type: NNE_OP_TYPE_Other_Pooling ● Warning! [layer_%d] Unsupported AK_Pooling! pad_right=[%d].Set new type: NNE_OP_TYPE_Other_Pooling ● Warning! [layer_%d] Unsupported AK_Pooling! pad_bottom=[%d].Set new type: NNE_OP_TYPE_Other_Pooling
AK_Global_Pooling	● 格式需满足表 3-7。 ● 输入个数为 1，输出个数为 1。	● Error! Unsupported AK_Global_Pooling! Format error. ● Error! Unsupported AK_Global_Pooling! ((layer.input_num != 1) || (layer.output_num != 1))
AK_Flatten	● 格式需满足表 3-7。 ● 输入个数为 1，输出个数为 1。 ● Flatten 输出数据[n,c,h,w]满足[1,1,1,w]	● Error! Unsupported AK_Flatten! Format error ● Error! Unsupported AK_Flatten! ((layer.input_num != 1) || (layer.output_num != 1))
AK_Slice	● 格式需满足表 3-7。 ● 输入个数为 1，输出个数为 2。	● Error! Unsupported AK_Slice! Format error. ● Error! Unsupported AK_Slice! ((layer.input_num != 1) || (layer.output_num != 2))
AK_Reshape	● 输入个数为 1，输出个数为	● Error! Unsupported AK_Reshape!

Anyka Confidential For Giec Use Only

算子	算子的参数约束	报错信息
	1。 ● 当出现在网络最后时，将被删除，提醒用户在后处理函数中处理。	((layer.input_num != 1) || (layer.output_num != 1)) ● Warning!!! Reshape(last layer) will be removed. Please implement it in PostProcess Function.
AK_Softmax	● 输入个数为 1，输出个数为 1。 ● 当出现在网络最后时，将被删除，提醒用户在后处理函数中处理。不支持网络中间出现 Softmax。	● Error! Unsupported AK_Softmax! ((layer.input_num != 1) || (layer.output_num != 1)) ● Warning! Softmax is the last layer. It will be removed from network. Please implement them in PostProcess Function!
AK_ShuffleChannel	● 格式需满足表 3-7。 ● 输入个数为 1，输出个数为 1。 ● group 必须为 2.reverse 必须为 0。	● Error! Unsupported AK_ShuffleChannel! Format error. ● Error! Unsupported AK_ShuffleChannel! ((layer.input_num != 1) || (layer.output_num != 1)) ● Error! Unsupported AK_ShuffleChannel! (group != 2) ● Error! Unsupported AK_ShuffleChannel! (reverse != 0)
AK_Split	● 输入个数为 1。	● Error! Unsupported AK_Split! (layer.input_num != 1)
AK_XRelu	● AK_Relu, AK_PRelu, AK_LeakyRelu,AK_Clip 统称为 AK_XRelu。 ● 输入个数为 1，输出个数为 1。	● Error! Unsupported AK_Relu! ((layer.input_num != 1) || (layer.output_num != 1))
AK_Concat	● 格式需满足表 3-7。 ● 输入个数最大值为 8，输出个数为 1。	● Error! Unsupported AK_Concat! ((layer.input_num > 8) ‖ (layer.output_num != 1))
AK_Upsample	● 格式需满足表 3-7。 ● 输入个数为 1，输出个数为 1。 ● 将拆分为 resize_w 和 resize_h 两步。 ● 最近邻模式，w/h 均放大 2 倍。	● NNE_OP_TYPE_Resize_H input/output size error!
AK_Add	● 格式需满足表 3-7。 ● 输入个数为 2，输出个数为 1。	● Unsupported AK_Add
AK_Padding	● 格式需满足表 3-7。 ● 输入个数为 1，输出个数为	● Error! Unsupported AK_Padding! Format error.

Anyka Confidential For Sec Use Only

算子	算子的参数约束	报错信息
	1。 ● w/h 维度 pad 值均为“0”时该算子被删除。 ● 当 AK_Padding 和 Ak_Pooling/AK_Other_Pooling 前后连接时合并。	● Error! Unsupported AK_Padding! ((layer.input_num != 1) || (layer.output_num != 1)) ● Warning! AK_Padding([0,0,0,0]) will be deleted. ● Warning! 'AK_Padding' and 'Ak_Pooling' will be merged.
AK_Gemm	● 格式需满足表 3-7。 ● 输入个数为 1，输出个数为 1。 ● 输入和输出数据维度均等于 2。input 和 weight 维度匹配。	● Error! Unsupported AK_Gemm! ((layer.input_num != 1) || (layer.output_num != 1))
	●	● Error! Unsupported AK_Gemm! input:[1,%d,%d],weight:[%d,%d]
AK_Transpose	● 格式需满足表 3-7。 ● 输入个数为 1，输出个数为 1。 ● 仅处理[n,c,h,w]维度，且 n==1.	● Error! Unsupported AK_Transpose! Format error. ● Error! Unsupported AK_Transpose! ((layer.input_num != 1) || (layer.output_num != 1))
其他	● 不支持的算子	● Error! Unsupported OP

convert_output_net.txt 文件中记录转换后的网络信息。其内容如表 3-9 所述。

表 3-9 convert_output_net.txt 信息说明表

第一行	印配置文件中关于输入的信息 input_w, input_h, input_c, input_color, input_norm。
第二行	存储 feature_map 的 buffer 的个数及每一个的 size。
第三行	网络参与运算的数据块的个数 N1
第四行~第 3+N1 行	一行表示一个 blob 的信息。包含 blob_idx、w、h、c、buffer_idx、is_input、is_output。
第 4+N1 行	网络层数 N2
第 5+N1 行 ~ 第 4+N1+N2 行	一行表示一层的信息。包含 layer_idx、op_type、input_num、input_blob_idx[8]、output_num、output_blob_idx[8]、后续信息因 op_type 不同而不同。
第 5+N1+N2 行	输入 blob 个数（N3）及对应的 blob_idx
第 6+N1+N2 行	输出 blob 个数（N4）及对应的 blob_idx
第 7+N1+N2 行	空行
第 8+N1+N2 行	network_input_blob:
第 9+N1+N2 行~ 第 8+N1+N2+N3 行	输入 blob 的信息，包含 name、blob_idx、buf_idx、w、h、c。
第 9+N1+N2+N3 行	network_output_blob
第 10+N1+N2+N3 行 - 第	输出 blob 的信息，包含 name、blob_idx、buf_idx、w、

Anyka Confidential For Gieq Use Only

9+N1+N2+N3+N4 行	h、c。blob 的顺序和 onnx 相同。如果某个 blob 信息和 onnx 有差异，可能是算子做了转换，例如“MatMul”将被转换为“Transpose+Conv+Transpose”，其中第二个“Transpose”满足“是输出层算子且使用 CPU 处理”的条件将被从主网络中排除，用户需在后处理函数中处理。在 convert 步骤的 log 中会提醒用户 output_blob 名字的改变。

convert_output_net_analyse.txt 文件中用另一种方式列出网络层信息，将输入/输出 blob 的 w/h/c 信息融入，并统计每一层的参数数量、MACs、读写数据量等信息及汇总信息。建议导入 excel 文档查看起来更清晰。

3.8 quan

quan 步骤输出定点模型。

3.8.1 量化方法介绍

量化的目的是将网络推理过程由浮点运算转换为定点运算，除法运算由“右移”操作替代。NNE 做卷积运算时数据类型为有符号 8 位整数，取值范围[-128, 127]。卷积参数及特征值由浮点数映射到[-128, 127]的整数区间，偏置（bias）由浮点数映射到 int 型，其他参数由浮点数映射到[-128, 127]的整数区间。映射过程采用线性映射，选取的放大倍数为 2 的指数倍。

对卷积参数的量化，按 kernel output channle 来对参数进行分组，每一组单独处理。对每一组数据，找到绝对值的最大值 max，将[-max, +max]之间的数据线性映射到[-128, 127]。

对特征值的量化方法命名为“NNE_QUAN_METHOD_Hist_Symmetry”。对校准图片集做正向推理过程中统计特征层数据的直方图分布，选取合适的数值区间，将浮点数映射到[-128, 127]。配置文件中的关键字“quan_hist_th”用来调节待映射的数值区间，取值范围[0,0.1]，如果不希望有数据被向上截断，设置为 0，典型值为 0.001、0.001、0.0001。建议用户分别尝试 0.01、0.001、0.0001、0.00001 并统计量化误差，选取量化误差最小的一组。

对特征值的量化方式还可设置为“NNE_QUAN_METHOD_USR_SET”，并把用户自定义量化参数写在该行的后面，用空格隔开。建议用户第一次使用量化工具时，设置为“NNE_QUAN_METHOD_Hist_Symmetry”，完成后得到“quan_output_HistTH_feature_shift.txt”，该文件列出了量化参数，在此基础上修改，第二次量化时设置为“NNE_QUAN_METHOD_USR_SET”，并把修改后的量化参数写在后面，重新执行量化步骤。可能出现的应用场景：优化检测网络后处理代码时，原本使用 softmax 计算 score 值，但想要用查表替代 softmax，此时需要确保每一个 score 层的量化参数保持一致且固定。那么可以修改量化参数中 score 层的参数确保满足要求。

某些层的量化参数可能会在统计后调整，如 concat，为保证两个输入的量化参数一致可能会调整其中一个，这里会带来误差。

量化顺序：先进行特征值量化，再进行参数量化。卷积参数中的 bias 由浮点数映射为 int 型数据，其放大倍数与 kernel 的放大倍数及输入特征值的放大倍数相关，因此需要先得到输入特征值的放大倍数。

Anyka Confidential For Gitee Use Only

3.8.2 工具

提供可执行文件，存放于：nne_compile_tool/Tools/Quan/quan。

3.8.3 输入文件

main 函数需要传入一个参数，该参数为配置文件名，与 convert 步骤所用配置文件为同一份。main 函数内会对该文件做解析。

其他输入文件已在配置文件中列出。

3.8.4 操作方法

命令行：

```
./quan    main_cfg_my_net.txt
```

请根据实际情况调整文件路径。

如果需要手动调整量化参数，则修改配置文件，将“quan_method”行修改为“NNE_QUAN_METHOD_USR_SET”，并在其后填写调整后的量化参数，再次执行以上命令。参考 nne_compile_tool/Demos/resnet_v2_50_retrain/main_cfg_UsrSet.txt.

3.8.5 输出文件

quan 工具输出 3 组文件，如表 3-10 所示。

表 3-10 quan 工具输出文件列表

序号	文件名	说明
1	xxx_fixed_model.bin	AK 定点模型。
2	quan_output_HistTH_feature_shift.txt quan_output_HistTH_feature_shift_pos.txt quan_output_param_shift.txt quan_output_param_feature_shift.txt	量化过程中的统计信息。分别为特征向量数值（绝对值）分布统计；特征向量数值（正数）分布统计；参数数值分布统计；参数和特征向量数值（绝对值）统计结果在同一文件中列出，方便分析。
3	quan_output_network_info.c quan_output_network_info.h	定点模型网络结构信息，以 C 代码形式保存。

通过 quan_output_network_info.h 和 quan_output_network_info.c 文件可以了解转换后的网络信息。

网络由多层“layer”组成，信息保存在 blob_info_all 数组中，数组个数表示“layer”数量，“layer”的信息用“NNE_LAYER_INFO_S”结构体来表示，结构体成员介绍如下：

```
typedef struct NNE_LAYER_INFO_S
{
    T_U16 layer_idx;        //层索引
    T_U16 op_type;          //层类型
    T_U16 input_num;        //该层输入 blob 的个数
    T_U16 output_num;       //该层输出 blob 的个数
    T_S16 input_blob_idx[8];    //该层输入 blob 的索引，最多 8 个输入，不足的填“-1”
    T_S16 output_blob_idx[8];   //该层输出 blob 的索引，最多 8 个输出，不足的填“-1”
    NNE_OP_ATTR_U op_attr;  //该层算子属性
}NNE_LAYER_INFO_S;
```

Anyka Confidential For Gtec Use Only

其中 blob 指数据块，例如“element_wise_add”算子输入 2 个 blob，输出 1 个 blob。对于网络推理过程中所有数据块都分配唯一的 idx，并记录详细信息。整个网络涉及的 blob 信息保存在 blob_info_all 数组中，数组数量表示 blob 数量，单个 blob 信息用 NNE_BLOB_INFO_S 结构体来表示，结构体成员介绍如下：

```
typedef struct NNE_BLOB_INFO_S
{
    T_U16 idx;          //blob 的索引
    T_U16 w;            //该数据块用 w,h,c 三个维度来表示，w 指 width 维度
    T_U16 h;            //该数据块用 w,h,c 三个维度来表示，h 指 height 维度
    T_U16 c;            //该数据块用 w,h,c 三个维度来表示，c 指 channel 维度
    T_U16 buffer_idx;   //存储该数据块的 buffer 的索引
    T_U16 is_input;     //该数据块是否是整个网络的输入数据块
    T_U16 is_output;    //该数据块是否是整个网络的输出数据块
    T_U16 quan_shift;   //该数据块的量化系数
}NNE_BLOB_INFO_S;
```

同一个 blob 可能会被不同算子复用。

输入或输出 blob 的最大数量为 8，不足时填“-1”。

网络推理过程中产生的数据（blob）保存在 DMA 内存中。工具根据网络结构、输入分辨率等信息统计所需内存的数量及大小，保存在 buffer_size_all 数组中。工具给每一层的输入和输出 blob 指定其存储的 buffer，用 buffer_idx 表示。

该层算子属性“NNE_OP_ATTR_U”是 Union 变量，成员介绍如下：

```
typedef union NNE_OP_ATTR_U
{
    OP_CONVLINE_ATTR_S  convline_attr;   //基本级联层信息
    TOP_POOL_ATTR_S pool_attr;           //“其他池化层”信息
    OP_PAD_ATTR_S pad_attr;              //Padding 层信息
    OP_SLICE_ATTR_S slice_attr;          //Slice 层信息
    OP_SLICE_ATTR_S transpose_attr;      //Transpose 层信息
}NNE_OP_ATTR_U;
```

每一种算子属性有对应的结构体表示，具体参考 quan_output_network_info.h。

3.9 check

check 步骤实现网络浮点正向推理和定点正向推理，需要用户提供后处理函数，该步骤可以检验前序步骤的正确性并评估量化前后精度。check 步骤所用的数据集可以和 quan 步骤所用量化数据集一致，也可以不相同。如果不相同，在 quan 执行完后，修改配置文件中修改 dataset_dir、dataset_list 两行，再执行 check 步骤。建议用户将 check 步骤所用数据集的一部分作为量化数据集。

3.9.1 工具

提供源码：

表 3-11 check.源代码文件列表

序号	文件名	说明	路径

序号	文件名	说明	路径
1	ak_nne_cbfun.h ak_nne_common.h ak_nne_net.h ak_nne_types.h	公共文件。	nne_compile_tool/Tools/Public/
2	parse_cfg.h parse_cfg.cpp	配置文件解析。 公共文件。	
3	ak_nne_check.h ak_nne_check.c	正向推理相关代码	nne_compile_tool/Tools/Check/
4	ak_nne_post_process.h ak_nne_post_process.c	用户实现的后处理文件	nne_compile_tool/Demos/xxx/vs_src/
5	main_check.cpp	用户实现的主函数	

3.9.2 输入文件

main 函数需要传入一个参数，该参数为配置文件名，与 convert 和 quan 步骤使用的是同一份。main 函数内会对该文件做解析。

其他输入文件已在配置文件中列出。

其中 draw_result 指定是否需要在图片上画出目标区域矩形框并保存图片，仅对检测网络有效。建议用户首次运行时，给 test_image_num 设置一个较小的值，并打开 draw_result，直观判断检测效果；当量化调试完毕，需要测试量化客观性能时，将量化结果保存到文档中，用于后续的精度分析，此时关闭画框保存功能可以提高速度。

3.9.3 操作方法

将 cnn_compile_tool/Demos/test_net/vs_src/ 目录下的 ak_nne_post_process.h，ak_nne_post_process.c 和 main_check.cpp 三个文件拷贝到 cnn_compile_tool/Tools/Check/ 目录下（如已存在则覆盖，注意保存），执行以下步骤：

```
cd cnn_compile_tool/Tools/Check/
mkdir build
cd build
cmake ..
make
```

在 cnn_compile_tool/Tools/Check/build/ 目录下生成可执行文件：check。将其拷贝到 cnn_compile_tool/Demos/test_net/vs_src/ 目录下，执行：

```
./check ../main_cfg_test_net.txt
```

3.9.4 输出文件

check 工程运行结束后会在 base_dir 目录下新建 result_float 和 result_quan 两个文件夹。result_float 用于存放浮点推理结果，结果保存在文档 result_float.txt 中，如果打开了 draw_result，画框后的图片也保存在此文件夹中。result_quan 用于存放定点推理结果，结果保存在文档 result_quan.txt 中，如果打开了 draw_result，画框后的图片也保存在此文件夹中。推理结果用于计算检测精度或识别精度，如需修改结果保存格式，可在 main_check.cpp 中修改。

For Internal Confidential Use Only

3.10 simulate

网络在开发板推理时要求使用固定的接口函数及数据结构，与前序步骤中在 PC 端的编码有所不同，simulate 工程用于从 PC 端到开发板的过度。在 simulate 工程中，使用与开发板相同的接口函数、数据结构、数据在内存中的排列方式、参数加载方式等。

3.10.1 工具

提供源码：

表 3-12 simulate 源代码文件列表

序号	文件名	说明	路径
1	ak_nne_cbfun.h ak_nne_common.h ak_nne_net.h ak_nne_types.h	公共文件。	nne_compile_tool/Tools/Public/
2	parse_cfg.h parse_cfg.cpp	配置文件解析。公共文件。	
3	ak_nne_driver.h ak_nne_driver.c	模拟实现 NNE 运算	nne_compile_tool/Tools/Simulate/
4	cb_fun.h cb_fun.c	模拟实现 svp 提供的回调函数	
5	ak_nne_forward.h ak_nne_forward.c	网络推理相关内容	
6	ak_nne_post_process.h ak_nne_post_process.c	用户实现的后处理文件	nne_compile_tool/Demos/my_net/vs_src/
7	main_simulate.cpp	主函数。测试一张图。	

3.10.2 操作方法

将源码列表中所列文件均拷贝到 cnn_compile_tool/Demos/test_net/vs_src/simulate 目录下。

将 cnn_compile_tool/Demos/test_net/vs_src/ 目录下的 ak_nne_post_process.h，ak_nne_post_process.c 和 main_simulate.cpp 三个文件拷贝到 cnn_compile_tool/Tools/Simulate/ 目录下（如已存在则覆盖，注意保存），执行以下步骤：

```
cd  cnn_compile_tool/Tools/Simulate/
mkdir build
cd build
cmake  ..
make
```

在 cnn_compile_tool/Tools/Simulate/build/目录下生成可执行文件：simulate。将其拷贝到 cnn_compile_tool/Demos/test_net/vs_src/目录下，执行：

```
./simulate ../main_cfg_test_net.txt
```

Anyka Confidential For Giec Use Only

3.10.3 输出文件

main_simulate.cpp 运行结束后，在 nne_compile_tool/Demos/test_net/debug/目录下生成测试图片的 rgbline 格式的输入数据及测试图推理结果文件。

3.11 上板运行

上板运行需要 libak_nne 库、libplat_npu 库和应用 demo。编译运行环境为 SDK 环境。

libak_nne 库保存在 nne_compile_tool/上板验证/libak_nne/，包含 xxx_alios 和 xxx_linux 两个文件夹，分别对应 alios 和 linux 系统。每个文件夹内的文件如表 3-13 所示。

表 3-13 libak_nne 库包含的文件

序号	文件	说明
1	ak_nne_cbfun.h ak_nne_common.h ak_nne_types.h	libak_nne 库对应的头文件
2	libak_nne.a libak_nne.so	libak_nne 库文件

libplat_npu 库保存在 nne_compile_tool/上板验证/libplat_npu/，libplat_npu 文件夹的文件如表 3-14 所示。

表 3-14 libak_nne 库包含的文件

序号	文件	说明
1	ak_npu.h	libplat_npu 库对应的头文件
2	libplat_npu.a libplat_npu.so	libplat_npu 库文件

应用 demo 保存在 nne_compile_tool/上板验证/board_demo/nne_demo/，nne_demo 文件夹的文件如表 3-15 所示。

表 3-15 nne_demo 包含的文件

序号	文件	说明
1	ak_nne_cbfun.h ak_nne_common.h ak_nne_types.h	libak_nne 库对应的头文件
2	ak_npu.h	libak_npu 库对应的头文件
3	ak_common_inner.h ak_mem_inner.h list.h osal_drv.h	其他头文件
4	ak_nne_post_process.h ak_nne_post_process.c	网络后处理
5	nne_test_fun.h nne_test_fun.c nne_demo.c	board_demo 主函数在 nne_demo.c 中，实现主要功能的函数在 nne_test_fun.c 中。
6	Makefile	Makefile

Anyka Confidential For Giec Use Only

3.11.1 编译应用 demo

编译应用 demo 所涉及的相关文件目录结构如图 3-6 所示。

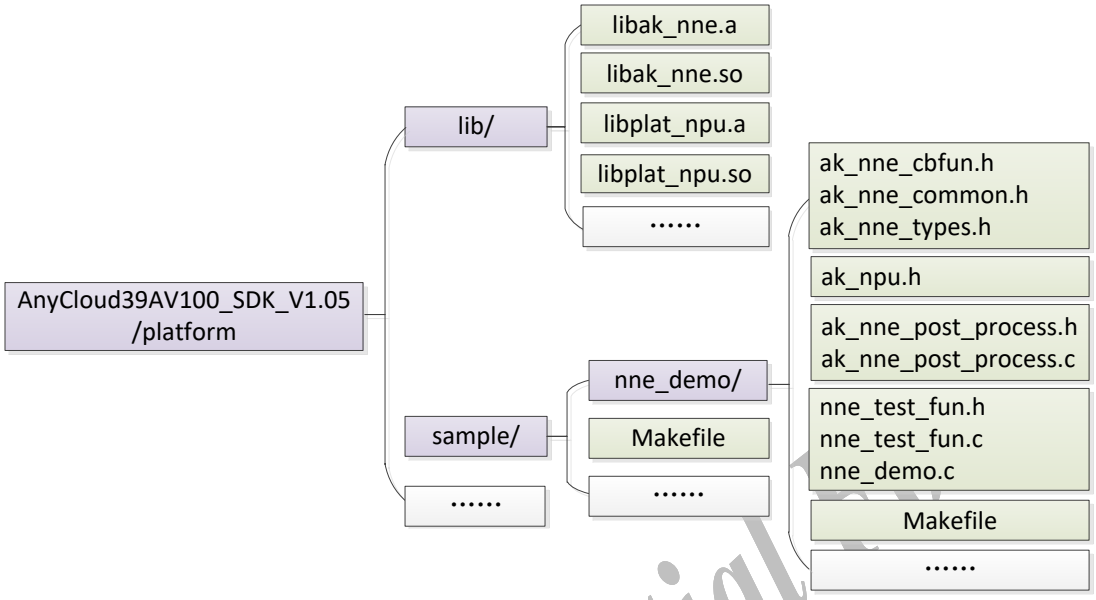


图 3-6 编译应用 demo 所涉及的相关文件目录

- step1：将 nne_demo 文件夹整体复制到 AnyCloud39AV100_SDK_V1.08/platform/sample/目录下。修改 AnyCloud39AV100_SDK_V1.08/platform/sample/路径下的 Makefile（与 nne_demo 文件夹同级），在文本 "dir_y += svp" 下行添加文本 "dir_y += nne_demo"。添加的文本中的目录名应与刚拷贝的文件夹 nne_demo 同名。示例如下：

```
ifeq ($(SVP), y)
    dir_y += svp
    dir_y += nne_demo
endif
```

- step2：将 libak_nne 库的头文件（ak_nne_cbfun.h、ak_nne_common.h、ak_nne_types.h）复制到 AnyCloud39AV100_SDK_V1.08/platform/sample/nne_demo/目录下。（nne_demo 目录下已存在，可不复制）。
- step3：将 libplat_npu 库的头文件（ak_npu.h）复制到 AnyCloud39AV100_SDK_V1.08/platform/sample/nne_demo/目录下。（nne_demo 目录下已存在，可不复制）。
- step4：将 libak_nne 库的库文件（libak_nne.a、libak_nne.so）复制到 AnyCloud39AV100_SDK_V1.08/platform/lib/目录下。
- step5：将 libplat_npu 库库文件（libplat_npu.a、libplat_npu.so）复制到 AnyCloud39AV100_SDK_V1.08/platform/lib/目录下。
- step6：修改 AnyCloud39AV100_SDK_V1.08/platform/sample/nne_demo/路径下的 Makefile，LIB_DEP 后添加需要链接的库名“ak_nne”和“plat_npu”。注意删除库名前的'lib'字符。（nne_demo 目录下的 Makefile 已经是修改过的）。

```
LIB_DEP = plat_common plat_log plat_mem plat_thread pthread plat_osal plat_dbg ak_nne plat_npu
```

Anyka Confidential Only

- step7：AnyCloud39AV100_SDK_V1.08/platform/sample/路径下执行 make 命令，即可在 AnyCloud39AV100_SDK_V1.08/platform/sample/目录下生成可执行文件 nne_demo。

libak_nne 库中不包含后处理部分，与具体网络无关。应用 demo 中包含主函数和后处理函数等，用户可修改。当前 nne_demo 已经支持了 nne_compile_tool/Demos/目录下所列的网络，每一个网络的后处理都在 ak_nne_post_process.h/ak_nne_post_process.c 中实现。如果需要增加新网络，则修改 nne_test_fun.c 文件中 ak_nne_demo_normal 函数，在该函数中添加新网络相关代码，同时在 ak_nne_post_process.c 中添加新网络的后处理函数，在 ak_nne_post_process.h 中添加对应函数声明，最后在 AnyCloud39AV100_SDK_V1.08/platform/sample/路径下执行 make 命令，重新生成可执行文件。

3.11.2 运行应用 demo

压缩包中 nne_compile_tool/上板验证/test_data/文件夹提供了 nne_compile_tool/Demos/路径下各个网络的测试数据，每个网络包含定点模型文件，待测试图片数据和该图片的测试结果。如 test_data/mobilenet_v1/中，mobilenetv1_fixed_model.bin 为 quan 步骤生成的网络定点参数文件，mobilenet_v1_224.rgbline 为待测试图片，result.txt 为该图片对应的输出结果。上板运行 demo 时，可使用测试图片验证 cnn 库和 demo 的编译运行是否正常。

以 mobilenet_v1 为例，将第 1 步骤生成的可执行文件 nne_demo、test_data/mobilenet_v1/目录下的 mobilenetv1_fixed_model.bin 和 mobilenet_v1_224.rgbline 拷贝到 TF 卡中。

1) 将开发板连接至 PC（串口转 USB 线），连接电源线。
2) 打开 xshell 或 WindTerm 新建连接。

```
协议：SERIAL
端口号：22
Port: （根据实际情况选择）
Baud Rate:115200
```

3) 给开发板上电，待打印完时输入用户名和密码。

```
anyka login: root
Password: （空）
```

4) 终止系统打印命令。

```
killall udhcpc
```

5) 挂载 TF 卡。

```
tf_card.sh
mount /dev/mmcblk0xx /mnt/
```

其中，mmcblk0xx 依 TF 卡而定。

6) 运行 demo。

```
cd /mnt/
./nne_demo mobilenet_v1 mobilenet_v1_fixed_model.bin mobilenet_v1_224.rgbline 224 224 3
```

输出结果可与 test_data/mobilenetv1/result.txt 中对比是否一致。

执行 nne_demo 时的各参数意义如下

```
./nne_demo    argv[1]    argv[2]    argv[3]    argv[4]    argv[5]    argv[6]
```

Anyka Confidential For Internal Use Only

表 3-16 开发板 Demo 命令行参数含义及示例

参数	含义	示例
argv[1]	网络名	mobilenetv1
argv[2]	AK 定点网络模型	mobilenetv1_fixed_model.bin
argv[3]	测试图数据	mobilenet_v1_224.rgbline
argv[4]	测试图 width	224
argv[5]	测试图 height	224
argv[6]	测试图 channel	3

其中网络名是为了区别后处理函数。这个应用 demo 可测试 11 个网络，而每个网络的后处理都不同，输入输出数据结构体也不同，所以需要一个标记。

该应用 demo 中添加了耗时统计代码，会输出“主网络推理”和“后处理”的耗时，例如：

```
d1=36109(us)        [NNE_Run]
d2=3042(us)         [post_process]
```

每个网络运行过程中会打印 DMA 内存占用情况。需要用到 DMA 内存的有两部分，分别是参数（param）和特征值（feature_map），以下是示例，单位是 Byte。

```
net_handle->hw_feature_buffer->size=1204224
net_handle->hw_param_buffer->size=4317620
```

Anyka Confidential For Giec Use Only